Republic of Tunisia
The Ministry of Higher Education and Scientific Research

The University of Carthage

The Higher Institute of Information and Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the

Bachelor Degree in Computer Science

Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE

Publicly defended on June 6, 2026 in front of the jury members:

Chairman:	Mohja Ben AMARA, SOFRECOM
Reporter:	Nour BRINIS, Professor, ISTIC
Examiner:	Moez ATTIA, Professor, ISTIC
Professional Supervisor:	Firas ABBESSI, Engineer, ITXTREE
Academic Supervisor:	Wassim ABBESSI, Professor, ISTIC

Academic Year: 2025-2026

Republic of Tunisia
The Ministry of Higher Education
and Scientific Research

The University of Carthage

The Higher Institute of Information and
Communication Technologies

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the

Bachelor Degree in Computer Science

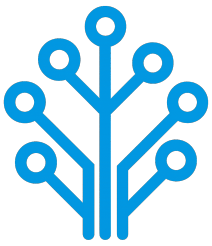
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

Amira BOUAFIF & Hajer JEMAA

Conducted within ITXTREE



Authorization of graduation project report submission:

Professional Supervisor:	Academic Supervisor:
Issued on :	Issued on :
Signature:	Signature:

Dedications

I dedicate this Work

To my dear parents

Dear mother Sawsen and father Elyes, you have been my greatest source of strength throughout this journey. Every sacrifice you have made, every prayer you have offered, and every word of encouragement you have given has brought me to where I am today. I hope to have made you proud, and I know that whatever challenges lie ahead, I will face them with your continuous support.

To my dear sisters and brothers

Oumayma, Eya, Yassmin, Mohamed and Yessin

Thank you for bearing with me through this period and for all the little things you did they mean to me more than the world. I am truly grateful to have you by my side, and I hope to give back the same love and support you have always given me.

To all my family and friends

To my family, thank you for your warmth and encouragement throughout this journey. To my friends, I may not have always said it, but I am deeply grateful for your presence in my life, and I hope to carry you with me for years to come.

Amira BOUAFIF

Dedication

In the name of Allah, the Most Merciful, the Most Compassionate.

I dedicate this work first and foremost to **Allah**, who is before everything and above everything. No one comes before Him, and nothing would have been possible without His guidance and mercy. The first words revealed to our Prophet Muhammad ﷺ were *Iqra*, read. This is a reminder of the great value Islam places on knowledge and learning, and of how seeking knowledge is a form of worship and closeness to God.

To my dear parents,

All gratitude is to you. I would not be who I am today without your love and sacrifices. From the day I was born until this moment where I stand today, everything I have achieved is because of you. I hope I will always make you proud and honor you the way you deserve.

My dear mother

You are a warm, positive, and caring person. Thank you for your endless love, your prayers, and your constant support that gave me strength in every step.

My dear father

You have always been by my side in my studies since my earliest years, even from the age of five. Thank you for your patience, your guidance, and your quiet but powerful support that never failed me.

To my dear sisters,

Asma, my dear sister, thank you for always being patient with me and for treating me with so much kindness and love.

Yosr, my dear sister, thank you for your presence in my life, for your support, and for always being there in your own special way.

To my grandmother, my uncles, aunts, and cousins, and to all my friends, thank you for your love, encouragement, and support throughout this journey.

And to everyone who believed in me in any way,
thank you all.

Hajer JEMAA

Acknowledgments

We would like to extend our sincere gratitude to all those who have supported us throughout the course of this project. Their guidance, encouragement, and support have been invaluable throughout this journey.

First and foremost, we would like to thank our academic supervisor, professor Wassim ABBESSI, for his guidance and insights, which have been a great help in the successful completion of this project.

Our gratitude also go to ITXTREE and, in particular, our professional supervisor, Mr. Firas ABBESSI, for providing us with the opportunity to work on this project and for their professional mentorship.

We also extend our deep appreciation to the professors at the Higher Institute of Information and Communication Technologies for their invaluable support and teachings, which have greatly contributed to our academic journey.

Finally, we thank everyone who has, in one way or another, helped in making this project a success.

Contents

List of Figures

List of Tables

Abbreviations

NIDS: Network Intrusion Detection System

AI: Artificial Intelligence

CRISP-DM: Cross-Industry Standard Process for Data Mining

UML: Unified Modeling Language

ICT: Information and Communication Technologies

SaaS: Software as a Service

SEO: Search Engine Optimization

iOS: IPod Operating System

CIC: Canadian Institute for Cybersecurity

UNB: University of New Brunswick

ACCS: Australian Centre for Cybersecurity

PCAP: Packet Capture

IAT: Inter-Arrival Time

NaN: Not a number

Inf: Infinity

DoS: Denial-of-Service

RAT: Remote Access Trojan

C&C: Command and Control

FTP: File Transfer Protocol

SSH: Secure Shell

DDoS: Distributed Denial-of-Service

XSS: Cross-Site Scripting

SQL: Structured Query Language

PCC: Pearson Correlation Coefficient

MI: Mutual Information

KNN: K-Nearest Neighbors

XGBoost: Extreme Gradient Boosting

ROS: Random Oversampling

SMOTE: Synthetic Minority Oversampling Technique

HTTP: Hypertext Transfer Protocol

TPE: Tree-structured Parzen Estimator

C2: Command and Control channel

GPU: Graphics Processing Unit

API: Application Programming Interface

REST: Representational State Transfer

SMTP: Simple Mail Transfer Protocol

CSS: Cascading Style Sheets

IDE: Integrated Development Environment

RDBMS: Relational Database Management System

PDF: Portable Document Format

General Introduction

In a world where nearly every aspect of life has become heavily reliant on interconnected systems, such as communication networks, banking systems, healthcare platforms, transportation systems, and even critical infrastructures, this growing dependence has, in turn, turned connectivity into a vulnerability to be exploited through cyber attacks, making cybersecurity a fundamental necessity.

And in the face of ever-growing and increasingly sophisticated threats, a race began. Among the many security mechanisms developed to address these threats, Network Intrusion Detection Systems (NIDS) play a critical role. As the name suggests, NIDS operate at the network level, acting as a key line of defense by monitoring network traffic and detecting malicious activities before they can further propagate into the system. However, the effectiveness of traditional NIDS is now challenged by the evolving nature of cyber threats. Built mostly around predefined rules and signatures, these systems remain rigid and reactive, making it difficult for them to keep pace with rapidly changing attack patterns.

With the rise of artificial intelligence (AI) and its integration into almost every domain, AI emerges as a promising solution to many complex problems, including cybersecurity, by introducing learning and adaptability capabilities.

In this context, Our project focuses on the design and implementation of an AI-based Network Intrusion Detection System following the **CRISP-DM**[1] methodology.

The first chapter, **«Project Context»**, is devoted to the presentation of the host organization, the analysis of existing solutions, and the definition of the problem statement with the proposed solution.

The second chapter, **«Requirements Specification»**, is dedicated to defining the functional and non-functional requirements, identifying the actors, and presenting the use case diagram.

The third chapter, **«Conceptual Design»**, presents the conceptual design of the system through the main UML[2] diagrams.

The fourth chapter, **«Data Collection and Preparation»**, covers data acquisition, data understanding, and data preparation following the phases of the CRISP-DM methodology.

The fifth chapter, **«Modeling and Evaluation»**, describes the adopted modeling approach, the training process, and the evaluation of model performance.

The sixth chapter, **«Realization»**, presents the development tools and technologies, model integration, deployment architecture, and the main user interfaces.

Finally, we conclude this report with a general conclusion.

Chapter 1

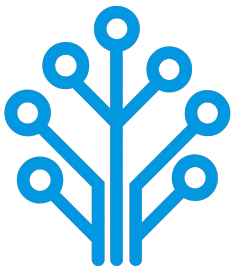
Project Context

1.1 Introduction

This chapter serves as an introduction to the project. We begin with a presentation of the company hosting the internship, followed by a study of existing network intrusion detection systems. Then, we present the problem and the proposed solution. Finally, we end the chapter with a conclusion.

1.2 Presentation of the company

Founded on January 28, 2025, ITXTree is a technology company operating in the Information and Communication Technologies (ICT) sector. The company positions itself as an innovation-driven player focused on designing scalable, forward-looking digital solutions. With a strong human-centered mindset, ITXTree aims to make artificial intelligence and advanced technologies more accessible to businesses across all industries.



1.2.1 Services and expertise

ITXTree's work is built around several key areas of expertise, enabling the delivery of end-to-end, tailored solutions:

- **SaaS and AI solutions:** Designing AI-powered applications to streamline operations and improve business processes.

- **Cross-platform software engineering:** Developing web and mobile applications (Android/iOS), as well as native desktop solutions.

- **Digital marketing and online visibility:** Providing SEO services to strengthen search engine ranking, indexing, and overall online presence for partners.

1.2.2 Vision and future outlook

ITXTree is on a path of continuous growth. After strengthening its capabilities in the domestic market, the company's next objective is to expand internationally. As part of its ongoing commitment to innovation, ITXTree also plans to move into cross-platform video game development in the near future, further demonstrating its ambition to broaden both its technical and creative expertise.

1.3 Study of the existing

Cybersecurity is a highly sensitive domain where the ability to react quickly can determine the impact of an attack. The earlier a threat is identified, the more effectively its consequences can be limited.

For this reason, Network Intrusion Detection Systems (NIDS) play a central role in monitoring and analyzing network traffic. The most widely used approaches are:

- **Signature-based:** These systems analyze network traffic, often through deep packet inspection and comparison against a signature database. When a match is found, an alert is triggered. Well-known tools such as Snort [3] and Suricata [4] are representative of this approach.

- **Anomaly-based:** Anomaly detection is learning the normal behavior of the network through statistical analysis of traffic parameters. When exceeding the predefined thresholds, it might be an indicator of an attack [5].

1.4 Criticism of the existing

Despite the widespread use of these systems, they have their own constraints. In particular, signature-based NIDS exhibit several limitations:

- **High Maintenance and Reactivity:** Their effectiveness is essentially tied to the completeness and accuracy of their signature rule sets. As attack techniques continuously evolve, these systems require frequent updates to remain effective. This maintenance process is both time-consuming and reactive since new signatures can only be created after an attack has already been identified and analyzed.

- **Performance:** As the number of rules increases, the system must process a larger set of comparisons for each packet, which can lead to slower detection. In a context

where reaction time is critical, this degradation directly affects the system's ability to respond effectively.

- **Rigidity and Evasion Vulnerability:** Attackers can exploit the rigidity of signature-based detection systems by introducing slight variations in attack patterns, allowing them to evade existing signatures while preserving the same malicious intent. Since many signature rules are publicly available or widely shared with the security community, attackers may analyze them to identify gaps and design variants that bypass detection mechanisms.

Overall, these systems rely on exact matching and predefined logic, which limits their adaptability in the face of evolving and increasingly sophisticated threats.

1.5 Proposed solution

With the growing integration of Artificial Intelligence in security systems, new approaches aim to overcome the limitations of traditional methods. We propose an AI-driven NIDS capable of learning patterns from network traffic and identifying different types of network activity instead of relying solely on predefined rules. The NIDS consists of training a multiclass classifier on a labeled network traffic dataset, and later integrating it into a web platform to enable continuous monitoring, as well as traffic analysis through dashboards, statistical summaries, and reporting.

1.6 Project pipeline

A project pipeline is a structured sequence of stages that outlines the workflow and processes involved in the development and deployment of a project. It serves as a roadmap, guiding the team through various phases, from initial planning to final delivery.

Figure 1.1 illustrates the project pipeline for our network intrusion detection system.

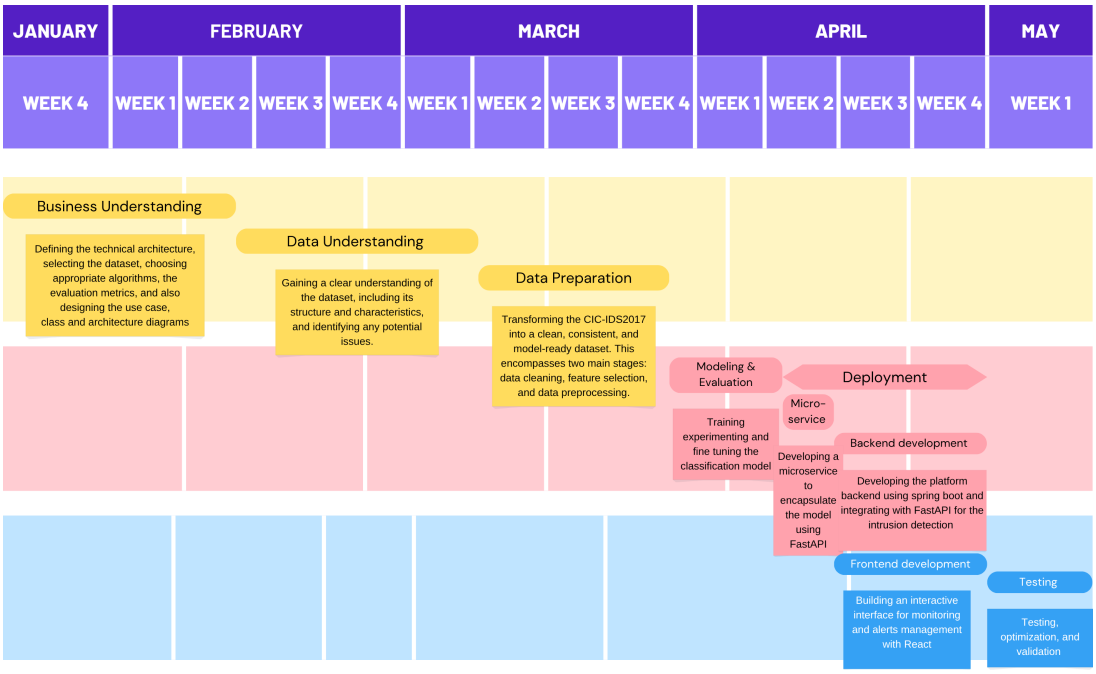


FIGURE 1.1 – Project pipeline

1.7 Methodology and modeling language

In this section, we present the **CRISP-DM**[1] methodology adapted in our project. We also define the system modeling approach, **UML**[2].

1.7.1 CRISP-DM

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a widely used framework for structuring data science projects. It consists of six phases:

- **Business understanding:** Defining project objectives and requirements and gaining a clear understanding of the business problem.
- **Data understanding:** Collecting and exploring the data to gain insights and assess its quality by identifying data-related issues.
- **Data preparation:** Cleaning and preparing the data for modeling.
- **Modeling:** Selecting and applying appropriate modeling techniques, building, testing, and tuning models to optimize their performance.
- **Evaluation:** Evaluating the model's performance and determining if it meets the defined objectives.
- **Deployment:** Deploying the model into production, monitoring and maintaining the model's performance.

Figure 1.2 illustrates these phases and their relationships.

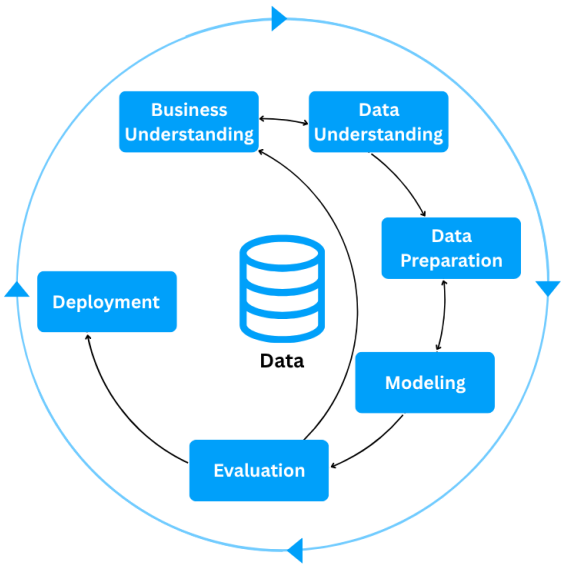


FIGURE 1.2 – CRISP-DM methodology

1.7.2 UML

For the system modeling, we use UML (Unified Modeling Language), a standardized modeling language used to specify, visualize, construct, and document the artifacts of software systems through diagrams such as use case, class, and sequence diagrams. There are two types of UML diagrams:

- **Behavioral diagrams:** State, activity, use case, sequence, communication and interaction overview diagrams.
- **Structural diagrams:** Class, object, component, deployment and package diagrams.

UML makes complex systems easier to design and communicate, it provides a common language for developers, architects, and stakeholders.

1.8 Conclusion

In this chapter, we introduce the host company ITXTREE, conduct a study of existing network intrusion detection systems, discuss their shortcomings and finally present our proposed solution. In the next chapter, we specify the system requirements.

Chapter 2

Requirements Specification

2.1 Introduction

Throughout this chapter, we specify both the functional and non-functional requirements, identify the system actors, and present the use case diagram. By defining these elements, we aim to establish a clear and shared understanding of the system's expected behavior and constraints, providing a solid foundation for the subsequent design and implementation phases.

2.2 Functional requirements

Functional requirements define *what* the system should perform. The main functionalities of our system are as follows:

— Authenticate;

— View Dashboard;

— View Statistics Summary;

— Trigger Network Analysis;

— View Alerts List;

— View Reports;

2.3 Non-Functional requirements

Non-functional requirements define *how* the system should perform its functions. They describe the quality attributes and constraints of the system:

- **Performance:** The system must process network traffic and generate analysis results with minimal latency, ensuring fast detection of potential intrusions.

- **Security:** The system must ensure secure authentication and enforce role-based access control, preventing unauthorized actions and protecting system integrity, while supporting non-repudiation to guarantee accountability and traceability of all operations.

- **Maintainability:** The system should be easy to maintain, allowing for efficient debugging, updates, and modifications to existing components.

- **Usability:** The system should provide an intuitive interface, enabling users to efficiently navigate and interact with its functionalities.

2.4 Actors identification

Actors are external entities, users, organizations, or external systems that interact with the system to achieve one or more functional requirements, with each actor representing a specific role.

In the context of our project, we identify two primary actors, as illustrated in Figure 2.1.

Figure 2.1 – Actors

Table 2.1 presents the roles of each actor in our system.

Table 2.1 – Roles of actors

Actor	**Role**
Network Administrator	- Authenticate - View Dashboard - View Statistics Summary - Trigger Network Analysis - View Alerts List
IT Manager	- Authenticate - View Dashboard - View Statistics Summary - View Reports

2.5 Use case diagram

A use case diagram is a visual representation of *how* users interact with the system and *what* the system does through different use cases. Figure 2.2 illustrates the diagram.

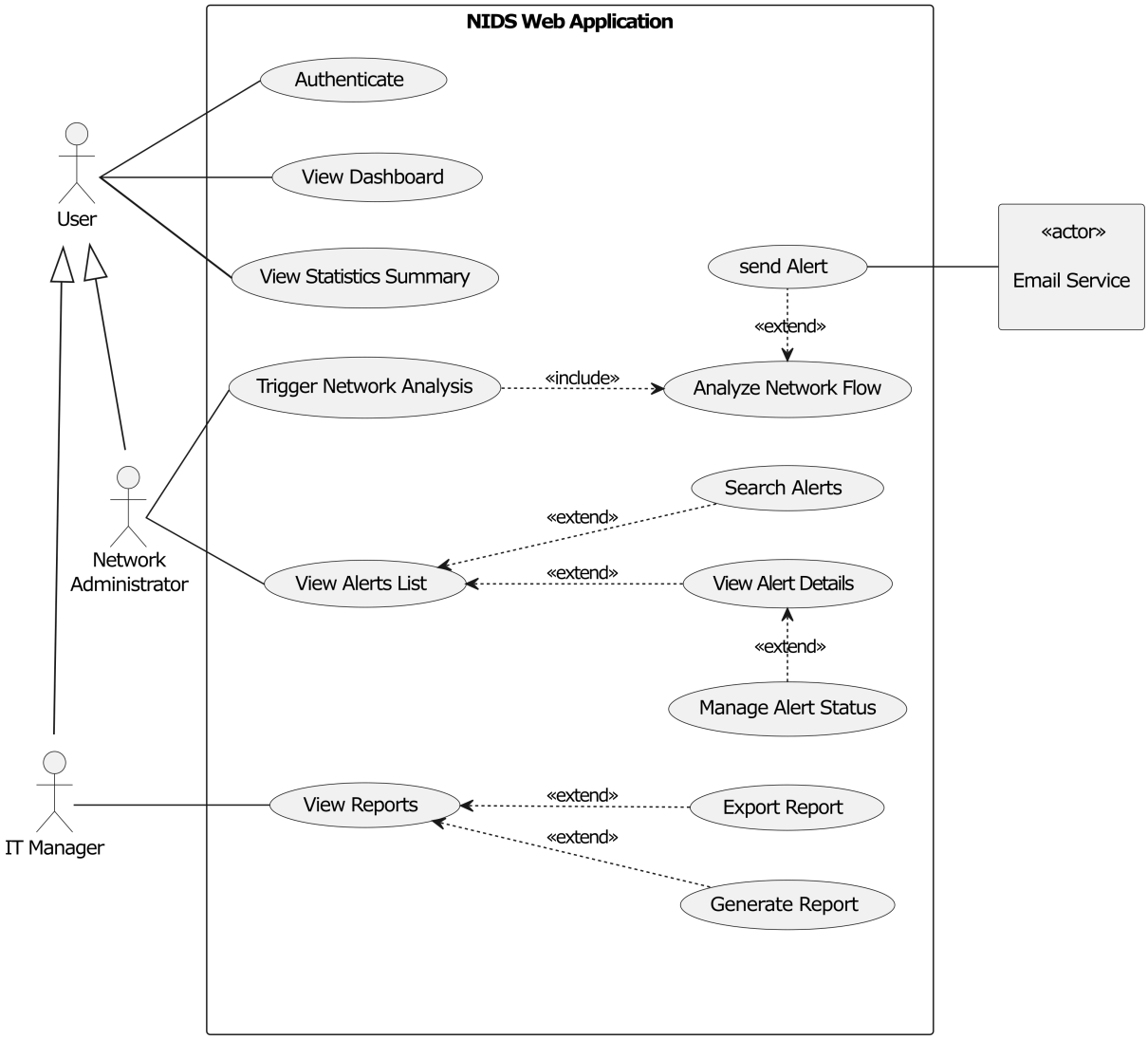


FIGURE 2.2 – Use case diagram

2.6 Conclusion

In this chapter, we outline the functional and non-functional requirements of our system, identify the primary actors, and represent the way they interact with the system and the system's functionality via a use case diagram. In the next chapter, we present the conceptual design of the platform.

Chapter 3

Conceptual Design

3.1 Introduction

This chapter focuses on the conceptual design of the application providing a clear and formal representation of both the dynamic behavior and the structural organization of the system, using the sequence diagrams, and the class diagram.

3.2 Sequence diagrams

A sequence diagram models the chronological interactions between system components for the most representative use cases.

In this section, two sequence diagrams are presented. The first covers the « Trigger Network Analysis » use case, which represents the core functionality of the platform. It models the full analysis pipeline: from the administrator triggering the simulation, through flow-by-flow prediction by the AI engine, to the conditional storage and alert generation based on the predicted label, including email notification handling for critical and high severity alerts. The second covers the « View Alerts List » use case, modeling the interactions involved in loading, paginating, filtering, and managing alerts, as well as navigating to alert details and updating alert status.

3.2.1 « Trigger Network Analysis » sequence diagram

Figure 3.1 illustrates the sequence diagram of the « Trigger Network Analysis » use case.

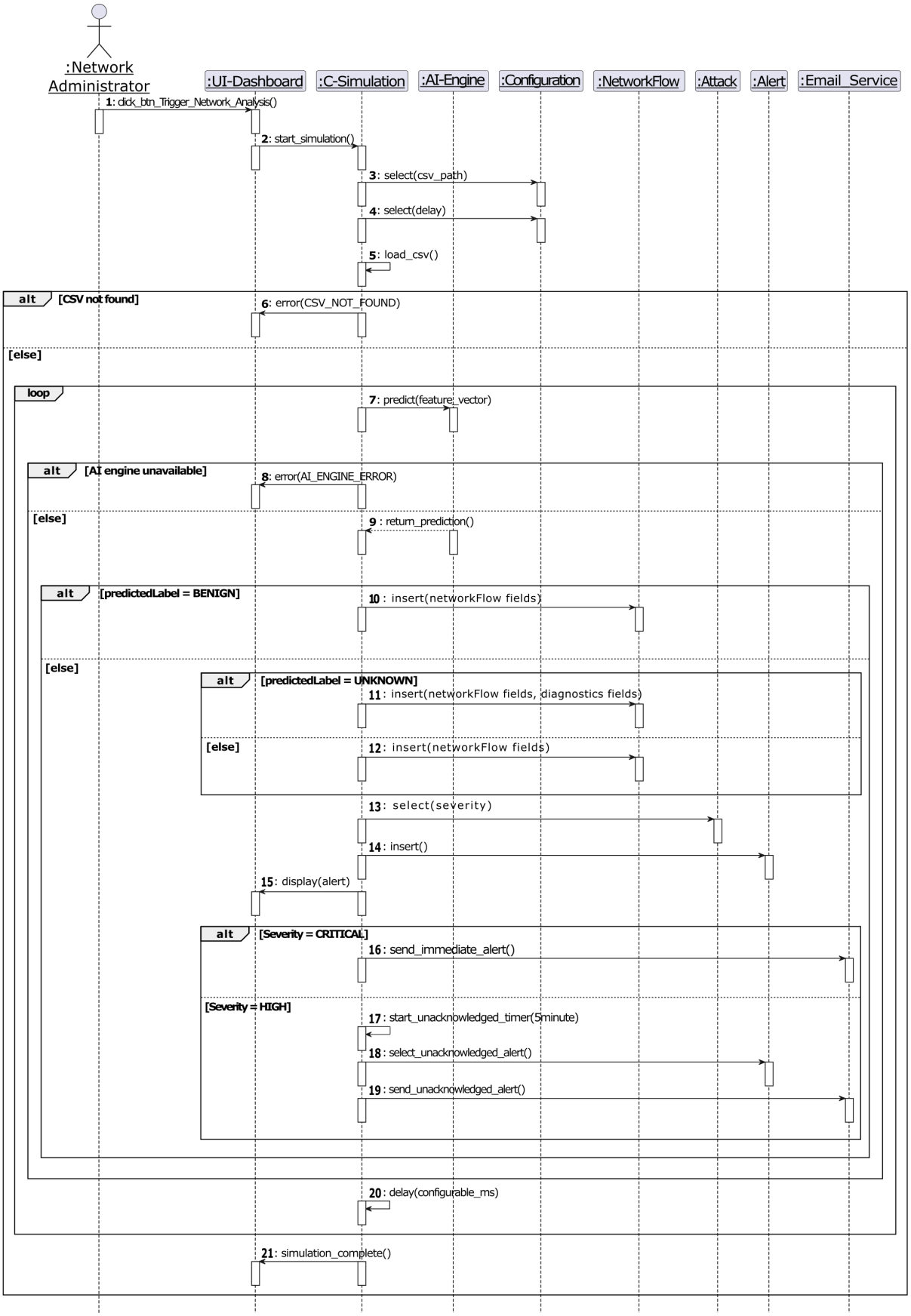


FIGURE 3.1 – Sequence diagram of the use case « Trigger Network Analysis »

3.2.2 « View Alerts List » sequence diagram

Figure 3.2 illustrates the sequence diagram of the « View Alerts List » use case.

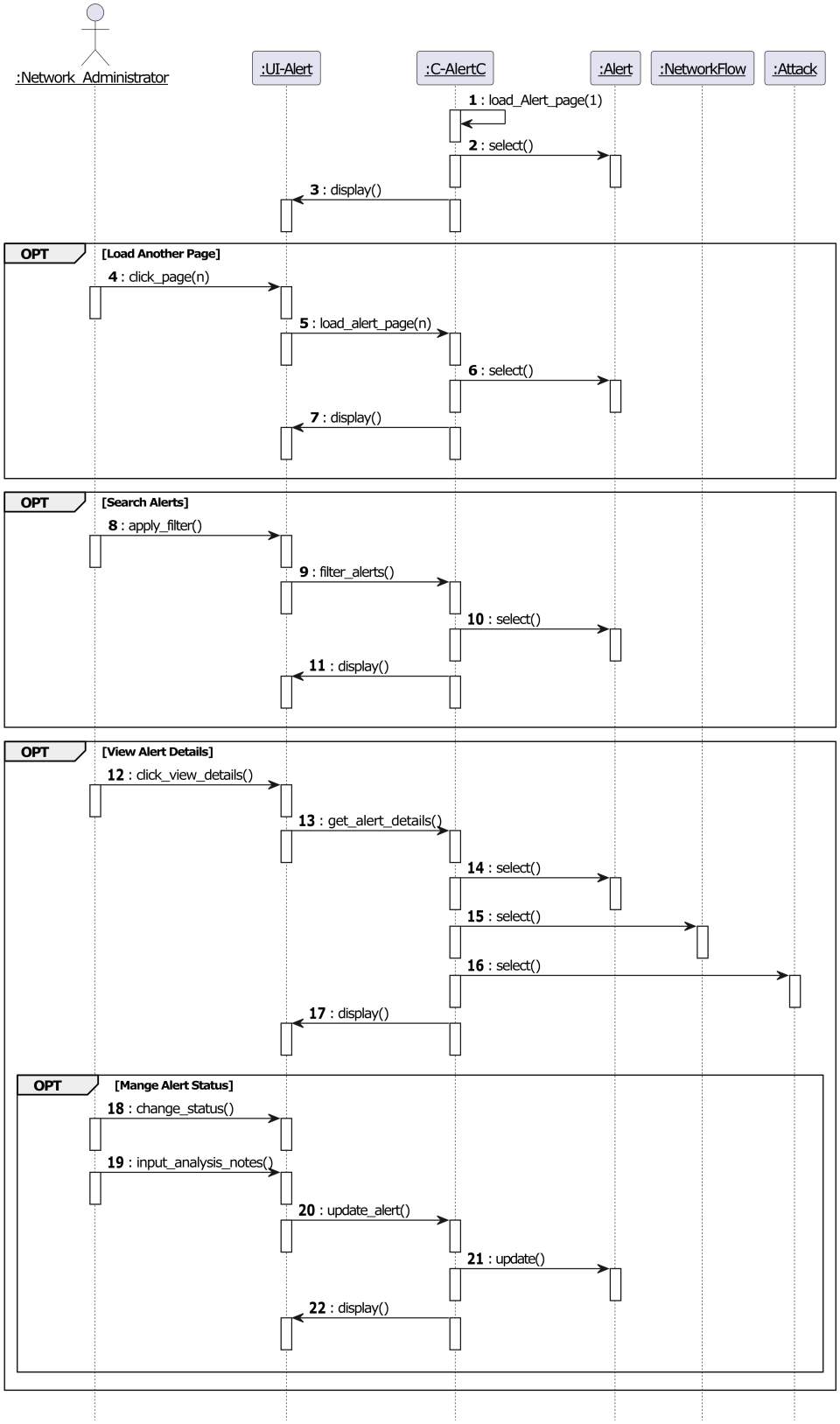


FIGURE 3.2 – Sequence diagram of the Use Case « View Alerts List »

3.3 Class diagram

A class diagram describes the static structure of the system through its classes, attributes, and relationships. Figure 3.3 illustrates the class diagram of our project.

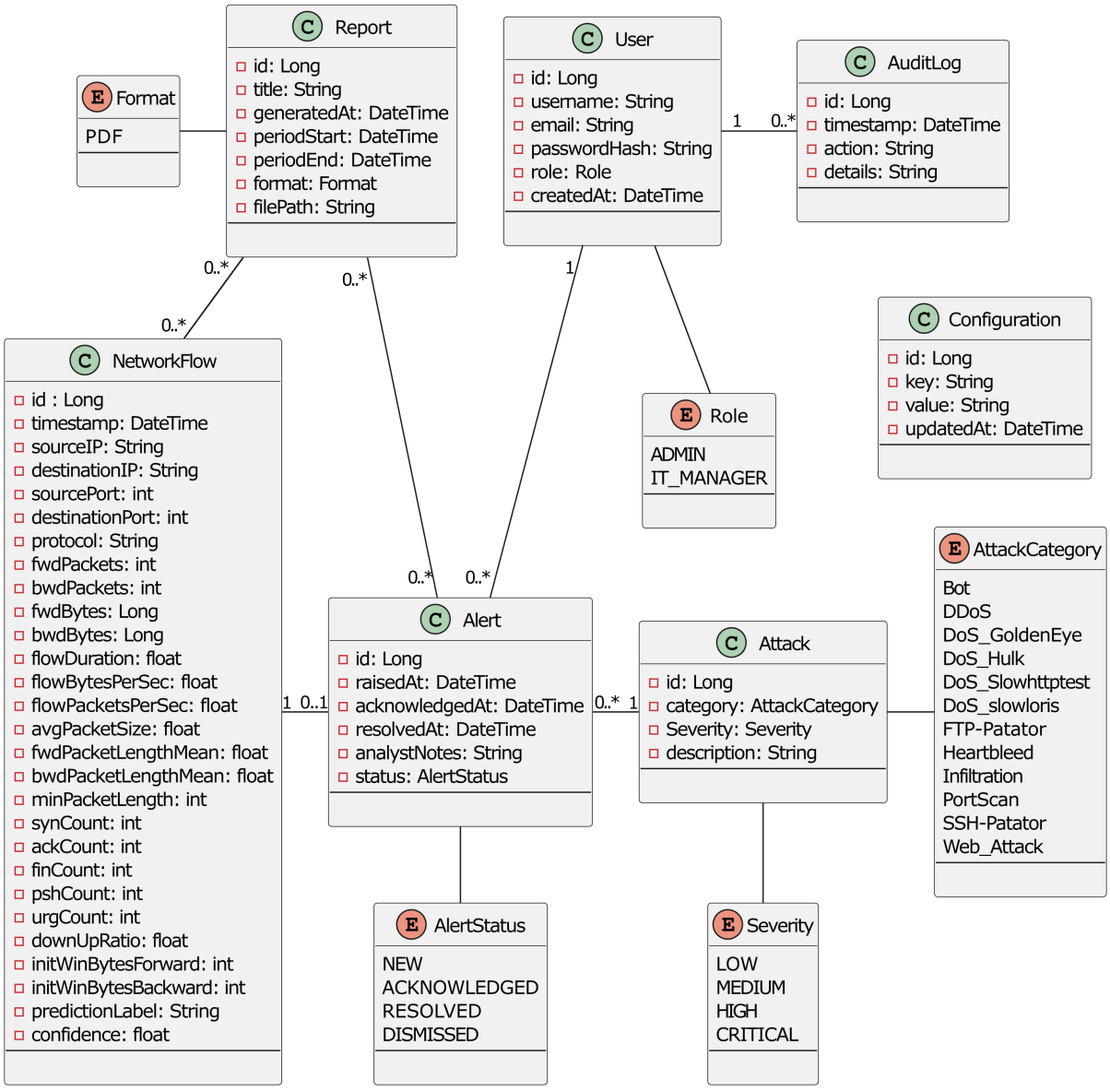


FIGURE 3.3 – Class diagram

3.4 Conclusion

In this chapter, we conclude the conceptual design phase using UML diagrams to model the system. The following chapter marks the first phases of the CRISP-DM methodology, data collection and preparation.

Chapter 4

Data Collection and Preparation

4.1 Introduction

Data constitutes the foundation of any AI system; the quality of all subsequent stages, including model training and performance, is directly dependent on it. For this reason, the data understanding and preparation phases of the CRISP-DM methodology are considered among the most critical stages of the process.

This chapter presents the complete data workflow adopted in this project. It begins with data collection, followed by an in-depth exploration phase aimed at understanding the structure, characteristics, and underlying patterns of the dataset. Based on the insights obtained during this stage, we then proceed to data preprocessing, where appropriate transformations and refinements are applied to ensure the dataset is properly structured and suitable for model training.

4.2 Data collection

This section introduces the process of acquiring the data used in this project. In general, this phase may involve either constructing a dataset from scratch or relying on existing sources. In our case, the decision to use a public dataset is mainly dictated by the nature of the cybersecurity domain.

Unlike other fields, network intrusion data cannot be collected on demand, as it depends on the occurrence of unpredictable and potentially harmful attacks. In addition, data collection in real operational environments is often constrained due to privacy concerns, security risks, and limited access to organizational traffic. Furthermore, even when such data is available, obtaining reliable ground truth labels is difficult and typically requires extensive manual analysis.

For these reasons, constructing a proprietary dataset is impractical, particularly in the absence of real network traffic provided by the host organization. Consequently, the use of a public dataset becomes a necessary and coherent choice, as such datasets are generated in controlled environments and include labeled instances based on expert-defined attack scenarios.

4.2.1 Dataset comparison

To decide on the dataset, we conduct a comparative analysis of available public datasets. We narrow it down to these three datasets. Below is a comparative Table 4.1 using our primary selection criteria.

Table 4.1 – Comparison of network intrusion detection datasets

Dataset	**Source**	**Size**	**Data Generation Approach**	**Attack Diversity and Temporal Relevance**	**Usability**
NSL-KDD [6]	Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB)	Small (≈148k records)	Derived from the KDD '99 dataset, its traffic was generated in a military-style network environment with predefined automated attack scripts.	Low; 4 attack families: DoS, Probe, R2L, U2R with several specific attacks in each family.	High; legacy benchmark widely used in academic studies, mainly for baseline comparison and educational purposes.
UNSW-NB15 [7]	Australian Centre for Cybersecurity (ACCS), UNSW Canberra	Medium (≈2.5M records)	Synthetic traffic generated with IXIA PerfectStorm tool.	Medium; 9 attack types: Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms.	High; widely used modern benchmark for machine learning-based intrusion detection research.
CIC-IDS-2017 [8]	Canadian Institute for Cybersecurity at the University of New Brunswick	Medium (≈2.8M records)	Traffic generated in an enterprise-like network environment composed of attacker and victim machines, using real attack tools.	High; 14 attack types: Includes the most common attacks based on the 2016 McAfee report, such as Web based, Brute force, DoS, DDoS, Infiltration, Heartbleed, Bot and Scan.	Very High; one of the most widely adopted and current standard datasets in intrusion detection research.

The comparative analysis of the three datasets reveals that each has its own limitations. NSL-KDD, although still relevant, exhibits limitations in size and temporal relevance, as it is relatively old and does not reflect recent network behavior and modern attack types. UNSW-NB15 is larger and offers a more recent alternative, but it remains limited in terms of attack type coverage, and its traffic is fully synthetically generated. This makes CIC-IDS2017 the most suitable dataset, as it provides a more realistic experimental setting, a wider range of the most common attack types, and it is also widely adopted in recent research studies.

4.2.2 Dataset overview

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity at the University of New Brunswick, in two phases. The first phase involved simulating network traffic within a controlled environment, where the generated traffic was captured in packet capture (PCAP) format. The second phase focused on transforming the raw PCAP files into a format suitable for machine learning. This transformation involved processing the captured network packets and aggregating them into network flows, from which statistical features describing each communication session were extracted using the CICFlowMeter [9] feature extraction tool. As a result, a row in this dataset does not represent a packet; it represents an overview of the session behavior between two endpoints through a statistical summary.

4.3 Data understanding

The Data Understanding phase of the CRISP-DM methodology focuses on loading, describing, and analyzing the dataset to build a solid foundation for the rest of the project. This section dives into the four tasks of this phase: `Initial Exploration`, `Data Description`, `Data Exploration`, and `Data Quality`, with each task targeting an aspect of the data, from its structure and content to its quality.

4.3.1 Initial exploration

The CIC-IDS2017 dataset is captured over five days. It is distributed across 8 files split by day and session: `Monday`, `Tuesday`, and `Wednesday` each corresponding to a single file, while `Thursday` and `Friday` are split into AM and PM sessions. The `Friday PM` session is further divided into two files to isolate the two attacks, `DDoS` and `PortScan`.

4.3.2 Data description

We examine the data on a surface level by identifying its structure, features, target label, and general characteristics.

As mentioned in the previous subsection, the dataset is split into 8 files as shown in Table 4.2.

Table 4.2 – Dataset files

File	Number of Rows	Number of Columns
Monday	529,918	79
Tuesday	445,909	79
Wednesday	692,703	79
Thursday AM	170,366	79
Thursday PM	288,602	79
Friday AM	191,033	79
Friday PM (DDoS)	225,745	79
Friday PM (PortScan)	286,467	79
Total	2,830,743	—

The dataset contains a total of 2,830,743 rows across the 8 files. All files have consistent columns, which is expected as they are generated by the same CICFlowMeter processing pipeline. This consistency allows for seamless concatenation of the files into a single DataFrame for the subsequent analysis.

Dataset formatting and label corrections

- **Whitespace issue:** Leading whitespace is found in column names and is stripped across all files, as it can raise `KeyError` exceptions when accessing columns by name.
- **Corrupted labels:** Corrupted labels are discovered in the Thursday AM file, where three labels contain mojibake characters instead of an em-dash (-), caused by an encoding issue, as shown in Figure 4.1, and are therefore corrected.

```
[Thursday-AM]  'Web Attack ◆ Brute Force'
[Thursday-AM]  'Web Attack ◆ XSS'
[Thursday-AM]  'Web Attack ◆ Sql Injection'
```

Figure 4.1 – Label corruption

Feature description

The dataset contains 79 columns, of them 78 are numeric level-flow features and the last one is the target Label. These features can be grouped into 12 categories as shown in Table 4.3.

Table 4.3 – Feature groups

Group	Features	Description
Flow Metadata	`Destination Port`, `Flow Duration`	Total duration of the flow and the destination port
Header Length	`Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`	Backward and forward header lengths.

Packet/Byte Count	`Total Fwd Packets`, `Total Backward Packets`, `Total Length of Fwd Packets`, `Total Length of Bwd Packets`, `act_data_pkt_fwd`	Number of packets and packet bytes in both forward (from source to destination) and backward directions (from destination to source), and packets having actual data in the forward direction.
Packet/Segment Length Statistics	`Fwd Packet Length Max/Min/Mean/Std`, `Bwd Packet Length Max/Min/Mean/Std`, `Max/Min Packet Length`, `Packet Length Mean/Std/Variance`, `Average Packet Size`, `Avg Fwd Segment Size`, `Avg Bwd Segment Size`	Packets and segments length statistics (e.g. max, min, mean, std etc...).
Flow Rate	`Flow Bytes/s`, `Flow Packets/s`, `Fwd Packets/s`, `Bwd Packets/s`, `Down/Up Ratio`	Flow throughput, forward and backward packet rates, flow asymmetry ratio (backward packets (download) to forward packets (upload)).
Inter-Arrival Time	`Flow IAT Max/Min/Mean/Std`, `Fwd IAT Max/Min/Mean/Std/Total`, `Bwd IAT Max/Min/Mean/Std/Total`	Time between consecutive packets in the flow, forward, and backward directions.
TCP Flag Count	`FIN Flag Count`, `SYN Flag Count`, `RST Flag Count`, `PSH Flag Count`, `ACK Flag Count`, `URG Flag Count`, `CWE Flag Count`, `ECE Flag Count`, `Fwd PSH Flags`, `Bwd PSH Flags`, `Fwd URG Flags`, `Bwd URG Flags`	TCP flags counts for the entire flow, forward and backward directions.
Bulk Transfer Statistics	`Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`	Average forward and backward bulk (a sequence of consecutive packets in the same direction with small inter-arrival times) transfer statistics.
Subflow Packet/Byte Count	`Subflow Fwd Packets`, `Subflow Fwd Bytes`, `Subflow Bwd Packets`, `Subflow Bwd Bytes`	Number of packets and packet bytes in both directions in a subflow.
Window/Segment Size	`Init_Win_bytes_forward`, `Init_Win_bytes_backward`, `min_seg_size_forward`	Window sizes in both the forward and backward directions and the minimal segment size in the forward direction.
Active/Idle Time	`Active Max/Min/Mean/Std`, `Idle Max/Min/Mean/Std`	Duration of active and idle periods within a flow.
Target	`Label`	The target label indicating the class the flow belongs to.

- **Duplicate column:** The column `Fwd Header Length.1` is identified as a duplicate of `Fwd Header Length`, confirmed by the fact that both contain the same values.

- `Packet Length Mean` and `Average Packet Size` are expected to be equal based on their naming and definition; however, they only match when the value is zero. They are defined as:

$$\texttt{Packet Length Mean} = \texttt{Average Packet Size} = \frac{\sum \text{Packet Length}}{\text{Packet Count}}$$

  We inspect the CICFlowMeter code source in order to trace back the computation logic of these two features. We found out that this discrepancy arises because `Packet Length Mean` uses a buggy version of the packet count variable, while `Average Packet Size` uses the correct one.

All 54 features are of type `int64`, 24 features are of type `float64`, with the last feature, the target *Label*, being of type `object`.

Table 4.4 presents the statistics of some of the features.

TABLE 4.4 – Feature statistics

Feature	Min	Max	Mean	Std
`Flow Duration`	-13	119,999,998	14,785,663.92	33,653,744.09
`Total Backward Packets`	0	121,922	10.39	997.39
`act_data_pkt fwd`	0	213,557	5.42	636.43
`Min Packet Length`	0	1,448	16.43	25.24
`Flow Bytes/s`	-241,000,000	inf	inf	nan
`Flow IAT Max`	-13	120,000,000	9,182,475.32	24,459,539.25
`Bwd PSH Flags`	0	0	0	0
`Fwd Header Length`	-322,212,234,632	4,644,908	-25,997.39	21,052,857.87
`Bwd Avg Bulk Rate`	0	0	0	0

Some features have negative, inf and constant statistic of zero. Those features are further investigated in the data quality section.

Attack classes description

Getting to understand the target label is just as important as any other step in the data analysis process. In this dataset, the target requires domain knowledge to interpret, as it represents attack classifications, and understanding the nature of each one is essential, as it helps later explain modeling outcomes, interpret misclassifications, and justify preprocessing and modeling decisions.

Table 4.5 provides a description of the 14 attack classes of the dataset.

TABLE 4.5 – Attack classes

Class	Description
BOT	An attack carried out using the ARES botnet, a malware-based network of compromised computers built on a RAT (Remote Access Trojan) and botnet framework, which allows the attacker to take control of infected machines and coordinate them for malicious purposes. It first infects victim machines using phishing emails or malicious downloads, after which the infected machines connect back to the attacker's Command and Control (C&C) server. Finally, the attacker issues commands to all bots, that is, all infected machines, simultaneously [10].
Heartbleed	An attack that exploits the HEARTBLEED bug in the OpenSSL cryptographic library. The attacker sends malformed heartbeat requests demanding more memory than the actual payload, causing the server to return unverified memory contents, exposing whatever sensitive data is stored there. This allows the attacker to steal private encryption keys, credentials, and session tokens, enabling session hijacking and decryption of previously intercepted traffic [11].
Infiltration	A multi-stage attack in which the attacker delivers a malicious file via a download link, containing a backdoor. Once the victim executes it, the backdoor establishes a reverse connection granting the attacker remote control. The attacker then scans the internal network using tools such as Nmap to discover live hosts, open ports, and running services, mapping the network and identifying targets for lateral movement. Finally, the attacker attempts to escalate privileges to access further restricted resources and exfiltrate sensitive data [8].
FTP-Patator (File Transfer Protocol Patator)	A brute force attack using the Patator tool targeting FTP servers, an old unencrypted protocol that transmits data and credentials in plain text, using trial and error to guess credentials in order to gain unauthorized access [12].
SSH-Patator (Secure Shell Patator)	A brute force attack using the Patator tool targeting SSH servers, a secure encrypted protocol that encrypts the entire session including the login, using trial and error to gain unauthorized access to individual accounts [13].

DoS Hulk	A Denial-of-Service attack using the HULK tool, which is used to overwhelm web servers by generating a large number of unique randomized requests at irregular intervals [8].
DoS GoldenEye	A Denial-of-Service attack using the GoldenEye tool, which dynamically generates multiple parallel connections (multi-threaded) to a target URL while keeping them alive to exhaust the available connection slots, simultaneously forcing the server to bypass caching mechanisms and fully process every request [14].
DoS slowloris	A Denial-of-Service attack using the Slowloris tool, which enables a single machine to overwhelm a web server by opening multiple connections and keeping them open by sending partial requests (only headers) periodically, while requiring very low bandwidth, hence its name, as it operates in a low and slow manner [15].
DoS Slowhttptest	A Denial-of-Service attack using the SlowHTTPTest tool, which creates very slow connections by sending seemingly legitimate requests using low bandwidth and intentionally delaying their completion, thereby consuming the limited number of concurrent connections the server can handle [16].
DDoS (Distributed Denial-of-Service)	An attack used to overwhelm the target system with a flood of traffic from multiple sources, making it unable to respond to legitimate requests, potentially leading to a complete shutdown of the system [17].
PortScan	An attack used to discover open ports, and which services are running on them, in order to identify potential vulnerabilities that could be exploited [8].
Web Attack - Brute Force	An attack targeting web applications, using a forceful approach to crack credentials and gain unauthorized access. It can be carried out in many ways, from trying every possible credential combination, to using dictionary attacks with a list of common passwords, or a combination of both [18].

`Web Attack - XSS`	An attack in which malicious scripts are injected into trusted websites, by exploiting their input validation vulnerability, and executed by victims' browsers, which have no way to distinguish it from legitimate content [19].
`Web Attack - Sql Injection`	An attack in which the attacker injects malicious SQL queries into an input field, which the vulnerable application passes directly to the database without proper sanitization. The database then executes it, enabling the attacker to view data that he is not authorized to access, including data belonging to other users and any other data that the platform has access to, as well as modify or delete it [20].

4.3.3 Data exploration

The data exploration tasks cover the class distribution and feature correlation analysis in order to gain a deeper understanding of the dataset.

Class distribution analysis

The class distribution analysis is organized into three aspects: an overall view of the class balance, a per-file breakdown, and a detailed breakdown of the distribution among attack classes.

- **Overall class distribution:** Table 4.6 provides a comprehensive overview of the classes' distribution across the dataset.

Table 4.6 – Class distribution

Class	file	Count	Percentage
BENIGN	Monday	529,918	18.72%
	Tuesday	432,074	15.26%
	Wednesday	440,031	15.54%
	Thursday AM	168,186	5.94%
	Thursday PM	288,566	10.19%
	Friday AM	189,067	6.68%
	Friday PM (DDoS)	97,718	3.45%
	Friday PM (PortScan)	127,537	4.51%
	Total	**2,273,097**	**80.3%**
DoS Hulk	Wednesday	231,073	8.163%
PortScan	Friday PM (PortScan)	158,930	5.614%
DDoS	Friday PM (DDoS)	128,027	4.523%
DoS GoldenEye	Wednesday	10,293	0.364%
FTP-Patator	Tuesday	7,938	0.280%

SSH-Patator	Tuesday	5,897	0.208%
DoS slowloris	Wednesday	5,796	0.205%
DoS Slowhttptest	Wednesday	5,499	0.194%
Bot	Friday AM	1,966	0.069%
Web Attack - Brute Force	Thursday AM	1,507	0.053%
Web Attack - XSS	Thursday AM	652	0.023%
Infiltration	Thursday PM	36	0.0013%
Web Attack - Sql Injection	Thursday AM	21	0.0007%
Heartbleed	Wednesday	11	0.0004%
Total		**2,830,743**	**100%**

The BENIGN class dominates the dataset, accounting for 80.3% of all samples, while the remaining 15 attack classes collectively represent only 19.7%. This ratio is visualized in Figure 4.2.

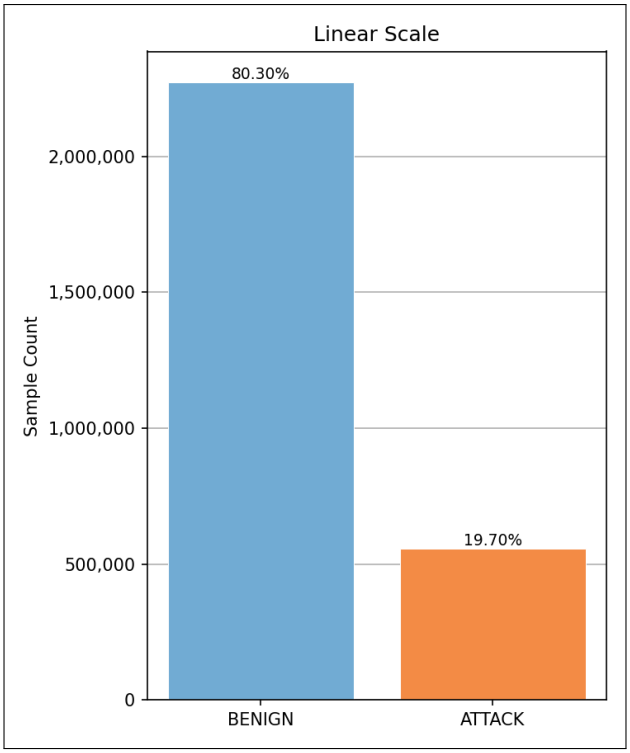


Figure 4.2 – BENIGN vs attack ratio

- **Class distribution per file:** The BENIGN class prevails across all files, with the only exceptions being the two Friday PM files, where attack traffic exceeds BENIGN, with attack ratios of 56.7% and 55.5% in the Friday PM DDoS and Friday PM PortScan respectively.

  Moreover, the BENIGN ratio ranges from 100% in the Monday file down to 43.3% in the Friday PM DDoS file. Several files show very low attack ratios, such as Thursday AM at 1.3% and Friday AM at 1%, while the Thursday PM file has an attack ratio as low as 0.01%, making it invisible in Figure 4.3 that shows the BENIGN vs Attack ratio per file.

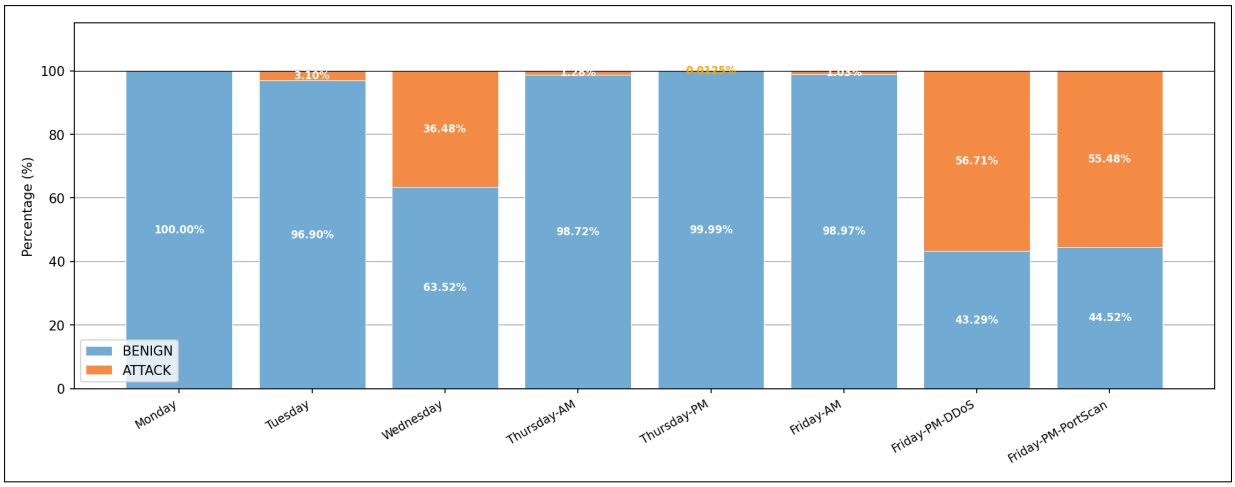


FIGURE 4.3 – BENIGN vs attack ratio per file

Each attack class is present in only one file, and each file contains between 0 and 5 attack classes, with most containing only one. The two brute force attacks, `FTP-PATATOR` and `SSH-PATATOR` are both present in the Tuesday file. Similarly, the `DDOS` attacks and `HEARTBLEED` are grouped in the Wednesday file, and the web attacks are present in the Thursday AM file. The remaining classes each appear exclusively in one of the other files. Figure 4.4 visualizes the class distribution per file.

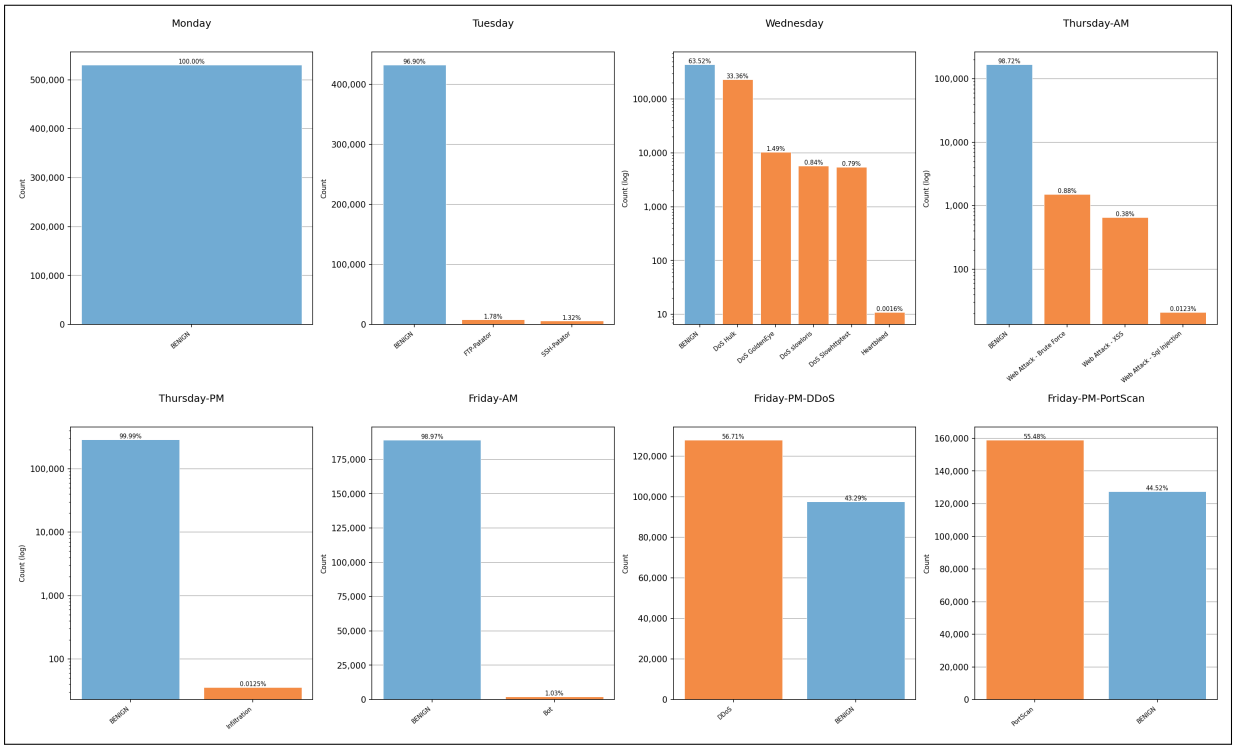


FIGURE 4.4 – Class distribution per file

- **Attack classes distribution:** The global attack distribution varies significantly, ranging from `DOS HULK` at 8.16% down to `HEARTBLEED` at 0.0004%, with the three rarest classes, `INFILTRATION`, `WEB ATTACK - SQL INJECTION`, and `HEARTBLEED`, collectively accounting for only 0.002% as shown in Figure 4.5.

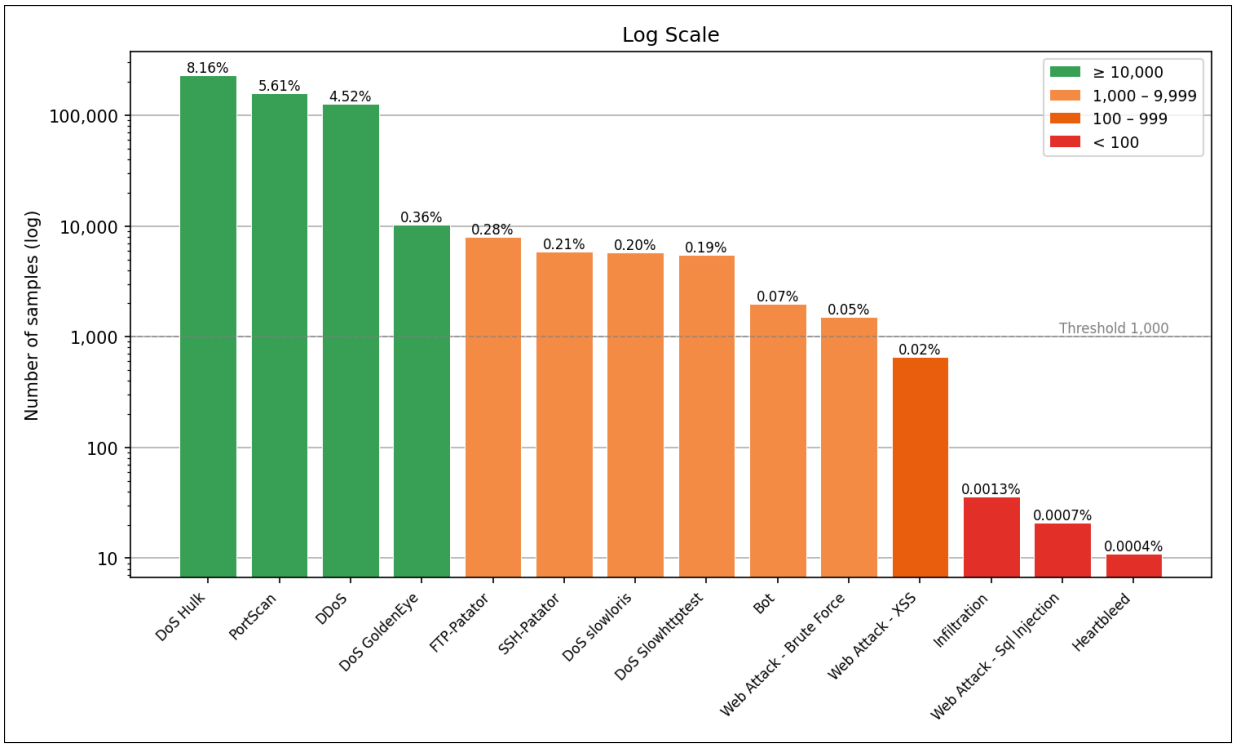


FIGURE 4.5 – Attack distribution

This distribution is intentional, as the dataset was generated to imitate real-world network traffic, where the vast majority of traffic is normal and only a small fraction are attacks. However, this introduces a severe class imbalance that can bias the model towards the majority class, leading it to treat `BENIGN` as the default outcome which may lead to attack classes being detected less frequently.

In addition, the class imbalance extends beyond the `BENIGN` class to attack classes themselves. `DoS Hulk`, `PortScan`, `DDoS` and `DoS GoldenEye` each exceed 10,000 samples, providing the model with sufficient examples to reliably learn their patterns. `FTP-Patator`, `SSH-Patator`, `DoS slowloris`, `DoS Slowhttptest`, `Bot` and `Web Attack - Brute Force` fall in a moderate range, each with over 1,000 samples, which, while on the lower end, should still allow the model to capture their general patterns.

The `Web Attack - XSS` class, however, with only 652 samples, sits in a more concerning range where learning may become difficult.

Finally, the three remaining classes represent the most critical imbalance: `Infiltration` with 36 samples, `Web Attack - Sql Injection` with 21 samples, and `Heartbleed` with only 11 samples, making them particularly challenging for the model.

Feature correlation analysis

The correlation matrix is computed for the 78 features of the dataset. It is a square matrix where each element represents the linear association between two features, with values ranging from -1 to 1. Positive values indicate that both features increase together, while negative values indicate that one increases as the other decreases. In both cases,

the strength of the correlation increases as the value approaches 1 or -1, while a value of zero indicates no correlation. The diagonal is always 1, as every feature is perfectly correlated with itself.

The correlation matrix is computed using the Pearson Correlation Coefficient (PCC), also known as Pearson's r [21]:

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

where x and y are the two features being compared, $\bar{x}$ and $\bar{y}$ are their respective means, and n is the number of samples.

The absolute value of the correlation matrix was taken to focus on the strength of the correlation regardless of its direction. A threshold of 0.95 was used to identify highly correlated feature pairs, where any pair with a correlation coefficient of 0.95 or higher is considered redundant, as both features carry the same information and therefore provide no additional value to the model.

Among the 78 numerical features in the dataset, 39 highly correlated pairs are identified. Their correlation matrix is visualized in Figure 4.6.

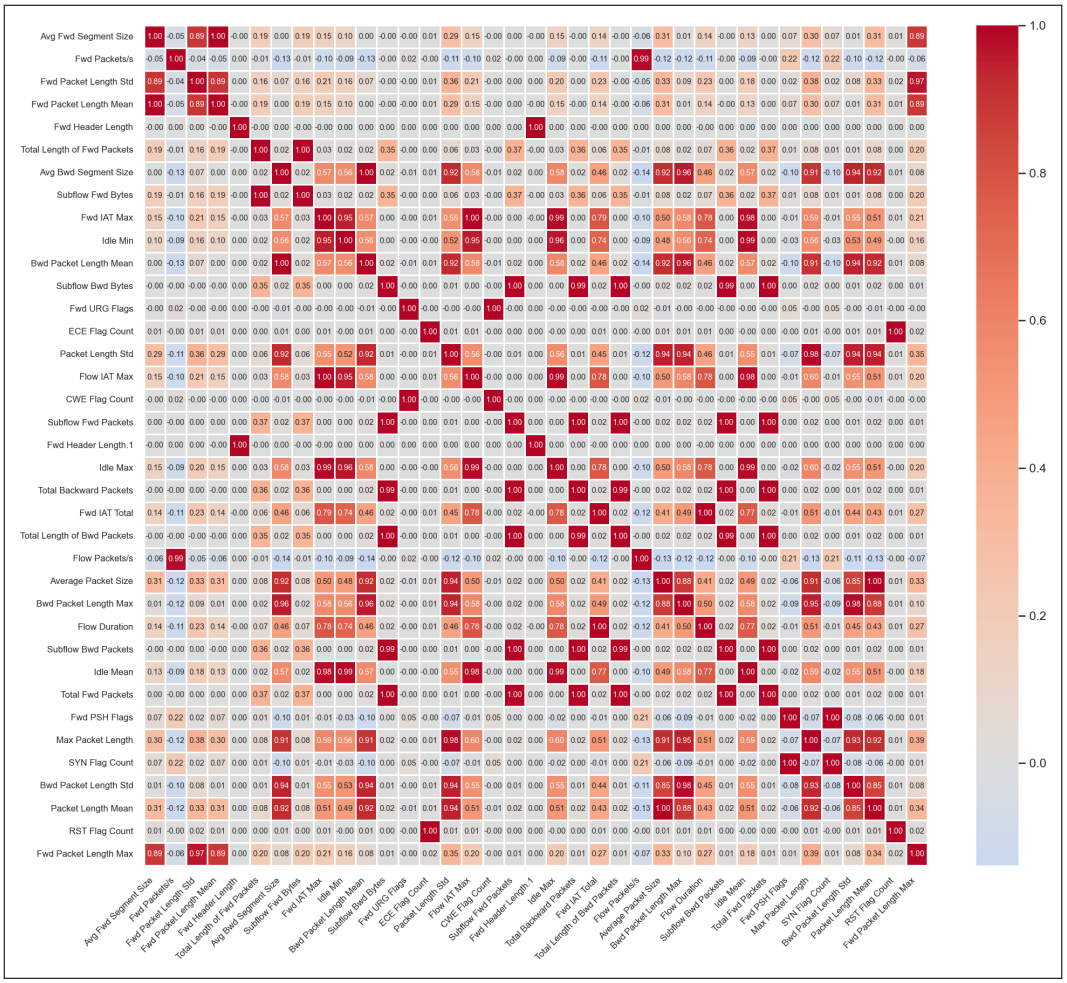


FIGURE 4.6 – Correlation heatmap

A perfect correlation coefficient of 1 is observed in eight pairs, as listed in Table 4.7.

Table 4.7 – Perfectly correlated feature pairs

No.	Feature 1	Feature 2
1	Bwd Packet Length Mean	Avg Bwd Segment Size
2	Fwd Packet Length Mean	Avg Fwd Segment Size
3	Total Fwd Packets	Subflow Fwd Packets
4	Total Backward Packets	Subflow Bwd Packets
5	Total Length of Bwd Packets	Subflow Bwd Bytes
6	Fwd PSH Flags	SYN Flag Count
7	Fwd URG Flags	CWE Flag Count
8	Fwd Header Length	Fwd Header Length.1

- `Bwd Packet Length Mean` and `Avg Bwd Segment Size`, `Fwd Packet Length Mean` and `Avg Fwd Segment Size`: going by name, the two calculate the average size of two different layers of the network, but in CICFlowMeter they both measure the average size (bytes) of the payload in the backward/forward direction, which means there is no difference between a segment and a packet, making the two features duplicates of each other.

- `Total Fwd Packets` and `Subflow Fwd Packets`, `Total Backward Packets` and `Subflow Bwd Packets`, `Total Length of Fwd Packets` and `Subflow Fwd Bytes`, `Total Length of Bwd Packets` and `Subflow Bwd Bytes` (near perfect correlation of 0.99999): Subflow features are intended to represent a finer level of aggregation within a flow, computed by dividing flow totals by the number of detected subflows. However, due to a bug in the CICFlowMeter source code, a misplaced parenthesis causes the subflow count to always equal 1, making subflow features identical to their flow-level counterparts. There is a minor discrepancy in the byte pairs; it is a type mismatch between `double` and `long`, affecting only flows with large byte totals.

- `Fwd PSH Flags` and `SYN Flag Count` are perfectly correlated throughout the dataset, taking only the values 0 and 1. Both are only updated on the first packet by CICFlowMeter, reducing them to binary indicators rather than true counters. The correlation is further reinforced by a traffic capture artifact: rows where both equal 1 belong exclusively to attacks that establish real TCP connections (FTP-Patator, SSH-Patator, DoS slowloris, DoS Slowhttptest, Infiltration) and benign traffic, while rows where both equal 0 correspond to attacks that bypass the handshake (DDoS, DoS Hulk, DoS GoldenEye, PortScan, Bot, Web Attacks) and midstream captures.

- `Fwd URG Flags` and `CWR Flag Count` are perfectly correlated throughout the dataset, with 2,830,428 rows where both equal 0 and only 315 rows where both equal 1, all belonging exclusively to benign traffic. As with the previous pair, the flag-counting bug reduces both features to binary indicators of the first packet only. The co-occurrence of URG and CWR (Congestion Window Reduced) on the same first packet is consistent with legitimate TCP congestion control behavior, which explains their absence in attack traffic, making both features redundant with each other.

- `Fwd Header Length` and `Fwd Header Length.1` these two features are duplicates of each other as mentioned previously in the Data Description subsection 4.3.2, which makes sense for them to be perfectly correlated.

4.3.4 Data quality

The quality of a model's output is directly tied to its input. In this section, we evaluate the dataset for any quality issues that could disrupt model training such as missing values (NaN - Not a number), infinite values (inf), negative values, duplicate rows, and constant and near-constant features. For each issue, the analysis investigates the root cause, assesses its global impact on the dataset across classes, and whether it endangers any minority classes.

NaN and infinite values

Scanning the dataset for undefined values revealed that NaN values appear exclusively in `Flow Bytes/s`, while infinite values appear in both `Flow Bytes/s` and `Flow Packets/s`. To understand the origin, the CICFlowMeter source code was inspected directly. The relevant formulas are:

$$\texttt{Flow Bytes/s} = \frac{\text{forwardBytes} + \text{backwardBytes}}{\text{flowDuration}/1{,}000{,}000}$$

$$\texttt{Flow Packets/s} = \frac{\text{packetCount (forward + backward)}}{\text{flowDuration}/1{,}000{,}000}$$

Both features divide by `Flow Duration`. When `Flow Duration = 0`, any nonzero numerator yields Inf, and a zero numerator yields NaN 0/0. This distinction maps directly onto two observable row types in the dataset:

- Rows with 2 forward packets, 0 backward, and nonzero byte count:
  `Flow Duration` $= 0$ with bytes > 0, producing `Flow Bytes/s` $=$ Inf.

- Rows with 1 forward packet, 1 backward packet, and zero byte count:
  `Flow Duration` $= 0$ with zero bytes, producing `Flow Packets/s` $=$ NaN.

The zero byte count in the second case, despite existing packets, is because CICFlowMeter computes `Total Length of Fwd/Bwd Packets` from the packet payload only. Packets carrying no data content only TCP headers (ACK, SYN) produce zero payload length. The true anomaly in both cases is `Flow Duration = 0`.

`Flow Duration` is computed as the timestamp difference between the last and first packet in a flow. It evaluates to zero when two packets arrive within the same timestamp resolution window as the tool lacks the granularity to distinguish arrivals that are nanoseconds apart. Every affected row contains exactly two packets total, either two forward or one forward and one backward, which is consistent with this explanation: with only two packets, a single resolution failure collapses the entire duration to zero.

This collapse propagates to every other feature that depends on the timestamp value, like `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, and all rate-based features reduce to zero as well.

The impact of these rows is relatively limited. Globally, 353 unique rows contain at least one NaN value, representing approximately 0.01% of the dataset, while 1,564 unique rows contain at least one Inf value, accounting for 0.05% of the dataset.

At the class level, Inf values mainly affect the `BENIGN` class, with 1,427 impacted rows corresponding to 0.06% of its samples, while only four attack classes are affected. NaN values are observed almost exclusively in `BENIGN`, with 350 affected rows representing less than 0.01% of the class samples, along with a small presence in `DoS Hulk`. Rare attack classes such as `Heartbleed`, `Infiltration`, and `Web Attack - SQL Injection` remain unaffected. The class-level distribution of affected rows is presented in Figure 4.7.

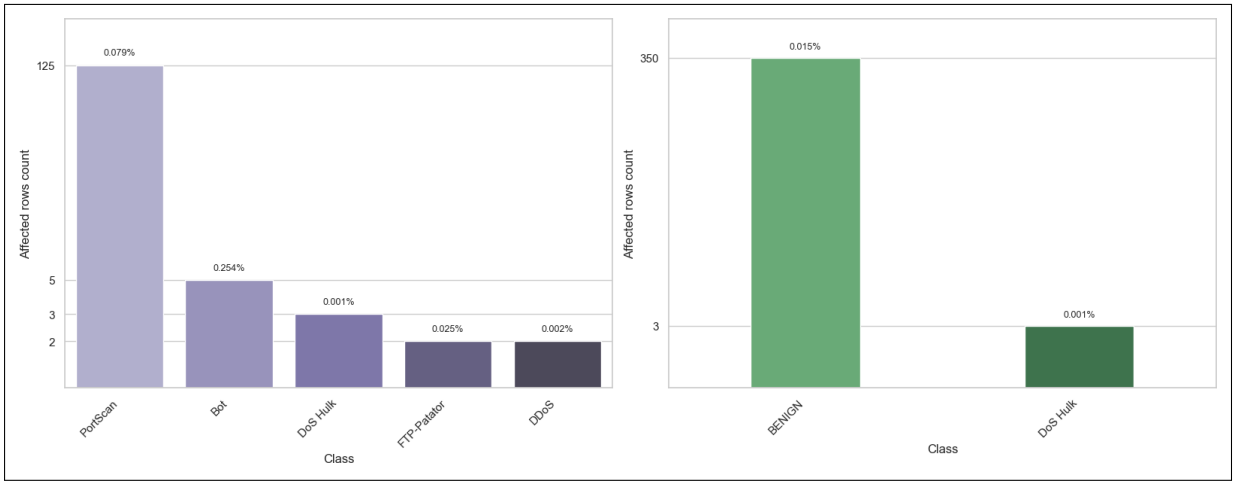


Figure 4.7 – Inf and NaN attack class distribution

Negative Values

Given the nature of the dataset features, their logical values should never be negative, yet negative values were found across several feature groups, each with a distinct cause.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

  These two features carry negative values at scale: 1,001,189 rows (35.4%) and 1,441,552 rows (50.9%) respectively hold the value -1. This is not a measurement error. CICFlowMeter uses -1 as a sentinel to signal that the TCP initial window size could not be captured, either because the SYN packet was absent from the capture, or because the flow is UDP-based and has no TCP handshake. It is a deliberate encoding choice by the tool, not a data corruption.

- **Packet ordering artifact:** `Flow Duration`, `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, `Fwd IAT Min`, `Flow Bytes/s`, and `Flow Packets/s`.

  A total of 115 rows carry negative values in `Flow Duration` ranging from -13 to -1, with the same negative values propagating identically into `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min`.

This is a cascade; since each of these rows has exactly one packet forward and one backward, there is only one inter-arrival interval in the flow, so mean, max, and min all equal `Flow Duration` exactly.

Division by a small negative duration scaled by 1,000,000 then causes `Flow Bytes/s` and `Flow Packets/s` to carry large negative values.
For example, with `Flow Duration` $= -1$ and 12 bytes of payload:

$$\texttt{Flow Bytes/s} = \frac{12}{-1/1{,}000{,}000} = -12{,}000{,}000$$

This is likely to be a packet ordering artifact when processing the packets; additionally, CICFlowMeter performs no guard against negative duration, so the error propagates unchecked. The affected rows are exclusively `BENIGN` traffic and represent less than 0.0001% of the dataset.

After excluding these 115 rows, an additional 2,776 rows exhibit negative `Flow IAT Min` values and 17 rows exhibit a negative `Fwd IAT Min`. Here the ordering artifact affects only a single packet pair within a longer multi-packet flow, dragging the minimum IAT negative while all other temporal features remain valid. The attack class-level distribution is shown in Figure 4.8.

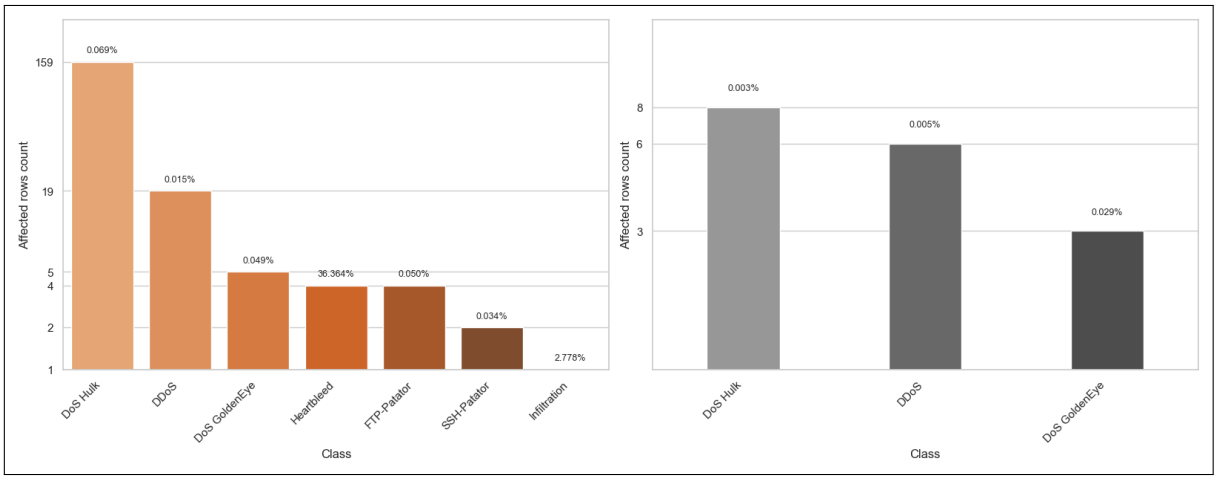


FIGURE 4.8 – Ordering artifact class distribution

- **UDP parsing bug:** `Fwd Header Length`, `Fwd Header Length.1`, `Bwd Header Length`, `min_seg_size_forward`.

  These features exhibit values in the range of -32,212,234,632 to -1,073,741,320, far outside any legitimate segment size and a TCP header size, which is bounded between 20 and 60 bytes.

  The root cause is a bug in CICFlowMeter: when processing UDP packets, the tool reads the header size from a TCP-specific field that simply does not apply to UDP, producing meaningless values that corrupt these features.

  The bug is confirmed by the data: all 35 affected rows are UDP flows, as indicated by `Init_Win_bytes_forward` = `Init_Win_bytes_backward` = -1.

The impact is narrow, 35 rows in `Fwd Header Length`, `Fwd Header Length.1`, and `min_seg_size_forward`, and 22 rows in `Bwd Header Length`, all exclusively `BENIGN` traffic, collectively under 0.002% of the dataset.

Exact duplicates

Duplicate rows are common in network datasets since a row in the dataset does not represent a packet but a session behavior summarized through statistical aggregation. Many flows share identical statistical profiles; thus, the presence of duplicates does not necessarily indicate a data error.

Globally, duplicate rows represent 308,381 out of 2,830,743 rows (10.89%). Their distribution by attack class is as shown in Figure 4.9, and no minority class is affected.

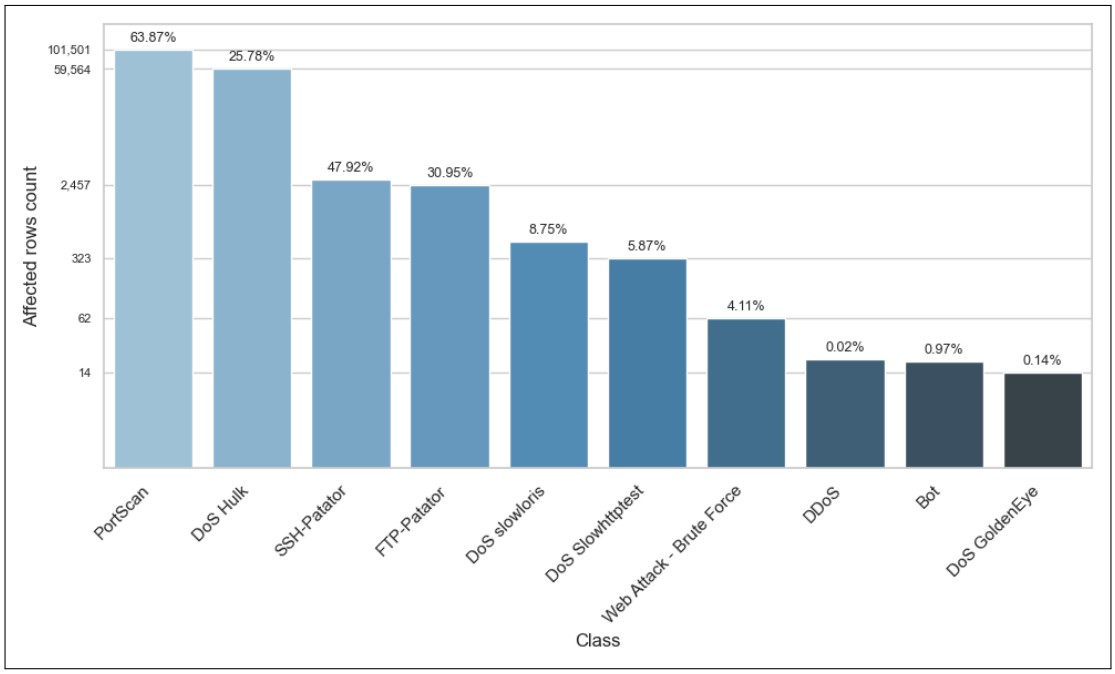


FIGURE 4.9 – Exact duplicates attack class distribution

Contradictory duplicates

These rows share the exact same feature values except for the target column, their labels differ. CICFlowMeter aggregates raw packet-level captures into flow statistics. Two flows from different sessions, one `BENIGN` and one malicious, can produce statistically identical feature vectors if their traffic patterns are indistinguishable at the flow-aggregation level. This is a fundamental limitation of statistical flow-based features rather than a labelling error.

Globally, 7,020 rows are affected. `BENIGN` traffic is the dominant participant in contradictory groups, as expected, `BENIGN` flows vastly outnumber attack flows, so the probability of a `BENIGN` flow sharing a feature vector with an attack flow is higher than inter-attack collisions. Contradictions involve only well-represented classes: `DoS Hulk`, `DDoS`, `DoS slowloris`, and `PortScan`. Figure 4.10 illustrates their distribution.

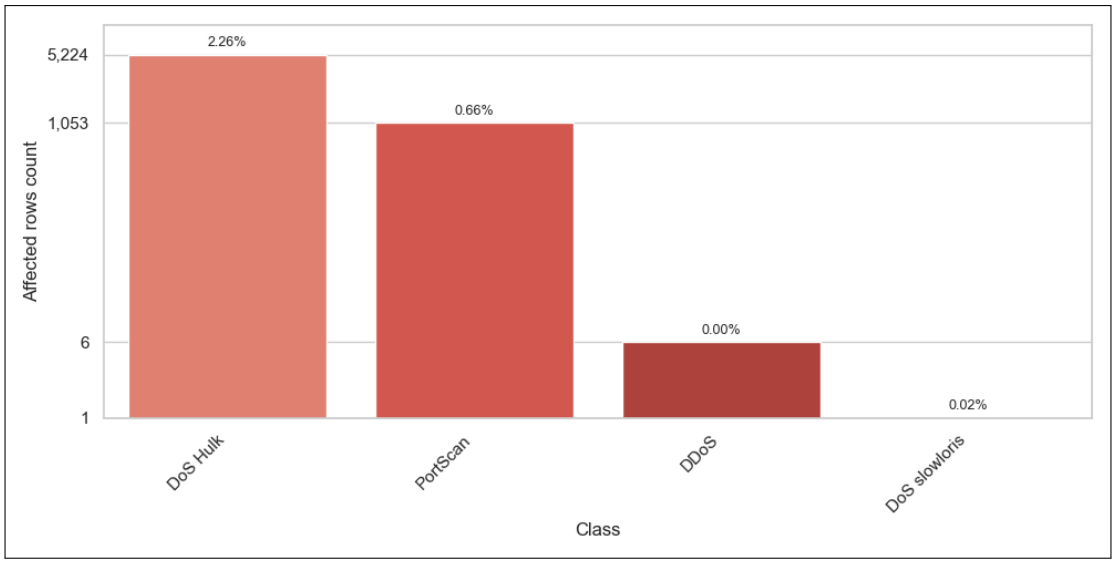


FIGURE 4.10 – Contradictory duplicates attack class distribution

Constant features

Constant features are those with a single unique value across the entire dataset, in this case, always zero. To understand this behavior, each feature was traced back to its computation logic in the CICFlowMeter source code. The root cause divides them into two groups.

- **Bulk transfer features:** `Fwd Avg Bytes/Bulk`, `Fwd Avg Packets/Bulk`, `Fwd Avg Bulk Rate`, `Bwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk`, `Bwd Avg Bulk Rate`.

  Bulk transfer refers to sending a large number of packets consecutively in one direction, with very little time between them.

  These features are derived from CICFlowMeter's bulk detection logic, which only activates when at least four consecutive packets carrying actual data are observed in close succession.

  If that condition is never met, either because flows in this dataset rarely involve this kind of continuous data sending, or because many packets carry no payload, the counters stay at zero and all six features are exported as zeros.

  This is exactly what is observed. These features are constant not by error, but because the bulk detection logic simply never triggers on this dataset.

- **Backward flag counters:** `Bwd PSH Flags` and `Bwd URG Flags`

  These are constant zero due to a bug in CICFlowMeter's implementation. The features `Bwd PSH Flags` and `Bwd URG Flags` are meant to count how many times the PSH and URG flags appear in packets traveling in the backward direction.

  These counters are however, only incremented inside the method that processes the first packet. Since the first packet by definition can only be forward, CICFlowMeter assigns the source IP from that first packet and compares subsequent packets against

it to determine direction. The method that handles the following packets never updates these counters, so both features remain zero for every flow, regardless of the actual traffic.

Near-constant features

`Fwd URG Flags`, `CWE Flag Count`, `RST Flag Count`, and `ECE Flag Count`.

Each feature holds exactly two unique values across the entire dataset. These are rare-event TCP control flags that are virtually absent in normal traffic. Combined, their non-zero occurrences account for only 315 rows (0.01%), all within `BENIGN` traffic. Therefore, they carry no discriminative signal for any attack class.

4.4 Data preparation

This section translates the findings from the data understanding phase into concrete preprocessing actions.

4.4.1 Data cleaning

The integrity of data is a prerequisite for any reliable model. Thus, we leverage data correctness over sample retention, as long as data volume and class balance are not meaningfully affected.

NaN and inf values

Rows carrying NaN or Inf values in `Flow Bytes/s` and `Flow Packets/s` are removed. As established, these arise from division by a zero `Flow Duration`, due to the lack of time granularity. Beyond the invalid rate features, these rows are also dominated by zeros which do not reflect the true nature of the flow, but are simply an artifact of insufficient measurement precision. Such a row cannot serve as a valid representation of a network session, as its defining characteristics were lost in the granularity gap.

The scale reinforces the decision. Globally, fewer than 0.05% of rows are affected. Additionally, the unique affected rows per attack class amount to 5 for `Bot`, 125 for `PortScan`, 2 for `DDoS`, and 3 for `DoS Hulk`, representing less than 0.3% of each of them, all of which carry sufficient clean samples to generalize from. No minority class is affected.

Negative values

Negative values across the dataset fall into four distinct categories, each handled differently.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

  These are retained without modification. As noted, -1 is a deliberate sentinel used by CICFlowMeter to encode the absence of a TCP handshake, it is semantically valid and carries information about the flow type. Replacing it would discard a meaningful signal.

- **Packet ordering artifact:** For the corrupted single-packet flow rows, in order to preserve data integrity and given the negligible rows count, only 115 belonging to `BENIGN` traffic, we decide to drop them.

  We clip invalid inter-arrival measurements in multi-packet flow rows to zero.

  The negative value appears only in the minimum IAT statistic, while `Flow Duration` remains positive and all other features are valid.

  Inter-arrival time cannot be negative by definition, so clipping both features to zero restores the value to the physical boundary it should never have crossed, without introducing artificial values.

  This targeted correction is justified precisely because the flow itself was captured correctly, unlike the single-packet cases above, where the corruption was total and removal was the only option.

  This treatment also preserves the 4 `HEARTBLEED` rows that would otherwise be discarded, a meaningful consideration given the class contains only 11 samples in total.

- **UDP parsing bug:** `Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`, and `min_seg_size_forward`.

  These rows are removed. As established, they are caused by a parsing bug rendering these feature values meaningless and irrecoverable. The fact that all affected rows belong exclusively to `BENIGN` traffic and represent under 0.002% of the dataset confirms that removal incurs no meaningful cost to class balance or dataset integrity.

Exact duplicates

Exact duplicate rows are dropped. Retaining them risks data leakage, a row could appear in both the training and test sets simultaneously, moreover it risks overfitting by biasing the model toward a single flow pattern. Since no minority classes are affected, the removal carries no risk to rare attack samples.

Contradictory duplicates

These rows present a serious risk to model training. An identical feature vector mapped to more than one target is unresolvable, retaining such rows forces the model to fit contradictory labels from the same input. All rows belonging to contradictory groups are therefore dropped entirely, regardless of their label. Since rare classes are unaffected, this carries no risk of losing minority attack samples.

4.4.2 Feature engineering

The quality of the feature space is just as important as the quality of the data itself. Redundant, constant, or non-discriminative features add noise and unnecessary dimensionality without contributing useful information to the model. Each feature removal decision in this section is grounded in either statistical evidence, domain knowledge, or both.

Constant features

All eight constant features are dropped. The bulk transfer group carries no information as the bulk detector is never triggered, making their true value zero throughout the dataset. The `Bwd PSH Flags` and `Bwd URG Flags` group share the same outcome, with the distinction that their zero values stem from a known CICFlowMeter implementation bug rather than a real measurement, leaving nothing to recover.

Near-constant features

`Fwd URG Flags`, `CWE Flag Count`, `RST Flag Count`, and `ECE Flag Count` are also dropped. These are not bugs, they are legitimate TCP control flags that appear in a small fraction of the dataset. However, with non-zero values appearing in under 0.03% of rows, exclusively in `BENIGN` traffic only, they carry no discriminative signal for attack detection.

Retaining features that offer no class-separating information adds unnecessary dimensionality without benefit, so they are removed to reduce dimensionality for a more focused feature space.

Feature correlation

The correlation matrix is recomputed on the cleaned data as the composition of the dataset has changed. 13 new pairs emerge and 3 pairs no longer appear, making a total of 49 pairs of highly correlated features, all of which provide no additional information to the model, some of which even have bugs in their computation causing the correlation.

The absence of the 3 pairs is explained by the cleaning process: `Fwd URG Flags` and `CWE Flag Count`, `RST Flag Count` and `ECE Flag Count` are near-constant features, and `Fwd Header Length` and `Fwd Header Length.1` are duplicate columns. As these features are no longer present in the dataset, the correlated pairs they formed no longer appear in the recomputed correlation analysis.

For the 13 new pairs, each pair contains at least one of the following features: `Fwd Header Length` and `Bwd Header Length`. These two features have their UDP parsing-corrupted rows dropped during cleaning. As a result, the true underlying correlation between these features and others becomes visible, previously masked by the corrupted values.

One feature from each pair is dropped. The choice is made based on Mutual Information (MI) score, which measures the feature-target association to determine which feature is more informative to distinguish between classes. For pairs with similar MI scores, the decision is made based on the domain knowledge.

Mutual Information measures how much knowing the value of a feature reduces uncertainty about the target class. Unlike the Pearson correlation, it captures both linear and non-linear relationships and works with categorical and continuous features. An MI score of 0 means it provides no useful information for target prediction, while a higher score indicates a stronger association with the target class.

The MI score measures the association between a single feature and the target independently of other features [22]:

$$MI(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

where X is the feature, Y is the target, $p(x,y)$ is the joint probability distribution of X and Y, and $p(x)$ and $p(y)$ are the marginal probability of X and Y independently.

For 37 pairs of the 49 highly correlated pairs, the feature with the higher MI score is retained. However, these decisions may be overridden by the Knowledge-based decision.

For the remaining 12 pairs the decision is made by examining what each feature represents in terms of network behavior rather than its statistical properties:

- **Packet and Segment length mean pairs:** `Fwd Packet Length Mean` and `Avg Fwd Segment Size`, `Bwd Packet Length Mean` and `Avg Bwd Segment Size`, the packets features are kept over the segments one, as they use standard network traffic analysis terminology, are immediately interpretable, and are consistent with the other packet length features already present in the dataset.

- **Totals and Subflows pairs:** `Total Fwd Packets` and `Subflow Fwd Packets`, `Total Backward Packets` and `Subflow Bwd Packets`, `Total Length of Fwd Packets` and `Subflow Fwd Bytes`, `Total Length of Bwd Packets` and `Subflow Bwd Bytes`, as established before, the subflows are equal to the totals throughout the dataset due to a bug in their computation. The totals are kept as they are correctly computed and more fundamental, representing the actual flow-level counts and byte totals directly. The subflow features are dropped as they provide no additional information beyond what the totals already capture.

- **Flags pair:** `Fwd PSH Flags` and `SYN Flag Count`, although both are only counted on the first packet due to the flag-counting bug, they still provide information about whether the flow was captured from the beginning of a TCP connection. Since both features carry identical information, the choice is based on domain relevance: `Fwd PSH Flags` only indicates whether the first packet had the PSH flag set, signaling the sender to push buffered data immediately, which carries limited interpretable meaning in the context of attack detection. `SYN Flag Count` is kept as it has a clearer semantic meaning, directly reflecting whether a connection-initiating SYN packet was observed, making it more interpretable for distinguishing attack types.

- **Duration and IAT pair:** `Flow Duration` and `Fwd IAT Total`, are highly correlated as both grow with the length of the connection, but they measure different temporal aspects. `Flow Duration` captures the total elapsed time of the entire flow across both directions and is retained as it is a direct, universally understood measure of connection length. `Fwd IAT Total` measures the cumulative inter-arrival time between consecutive forward packets only, making it a direction-specific partial measure whose information is already partially represented by `Flow Duration` in combination with the other IAT features, and is therefore dropped.

- **Idle multicollinearity pairs:** `idle mean` and `idle max`, `idle mean` and `idle min`, `idle max` and `idle min`, the latter pair's features are both dropped in favor of `Idle Mean`, the two represent the extremes of the idle time distribution and are jointly collinear with it. `Idle Max` is susceptible to inflation by a single unusually long pause without reflecting the overall idle behavior of the flow, while `Idle Min` is easily influenced by noise from brief transient gaps. `Idle Mean` is retained as it is more representative of the typical idle behavior across the flow and less sensitive to individual outlier values.

In conclusion, of the 33 features involved in the highly correlated pairs, 19 are dropped based on a combination of MI score and domain knowledge:

- `Subflow Bwd Bytes`
- `Subflow Bwd Packets`
- `Subflow Fwd Bytes`
- `Subflow Fwd Packets`
- `Packet Length Mean`
- `Avg Fwd Segment Size`
- `Avg Bwd Segment Size`
- `Bwd Header Length`
- `Fwd Header Length`
- `Bwd Packet Length Max`
- `Bwd Packet Length Std`
- `Fwd Packet Length Std`
- `Max Packet Length`
- `Flow IAT Max`
- `Fwd IAT Total`
- `Fwd PSH Flags`
- `Fwd Packets/s`
- `Idle Max`
- `Idle Min`

4.4.3 Label encoding

The selected model, XGBoost, requires numerical input for both features and the target variable. While all features are already numerical, the target column `Label` is categorical and stored as an object type. Hence, label encoding was performed using a label encoder from `sklearn.preprocessing`. The encoder was fitted and applied to the target column in a single step using `fit_transform()`. To preserve interpretability of the model's predicted outputs, the resulting integer-to-class mapping was exported as a `label_mapping.json` file derived from `le.classes_`, allowing any predicted label to be traced back to its original class name.

4.4.4 Train/Test split

Several factors make us approach this step with caution when assessing the train/test split strategies.

A per-file split is not an option, as the files are divided per attack type, each attack class appears in exactly one file, so splitting by file would result in classes entirely absent from either the training or test set.

A standard random 80/20 split presents its own issues. The CIC-IDS2017 dataset is heavily imbalanced, the `BENIGN` to attack ratio is approximately 80/20. Additionally, rare classes present less than 0.002% of the full dataset, so this strategy offers no guarantee of a fair class distribution and gives no assurance that rare classes will be adequately represented in both sets, which is particularly concerning given their scarcity.

For these reasons, a stratified train/test split is selected. It ensures all classes are present in both the training and test sets, each maintaining their original proportions, so the class distribution in each split mirrors that of the full dataset.

This strategy, however, does not solve the class imbalance. This issue will be addressed further during model training.

4.5 Conclusion

This chapter presented the full data workflow adopted for this project, from selecting an appropriate public intrusion detection dataset to preparing it for model training. We compared candidate datasets and justified the choice of CIC-IDS2017, then analyzed data quality issues such as NaN/Inf values, negative artifacts, duplicates, and constant features, linking each issue to its root cause in traffic generation or feature extraction. Finally, we applied cleaning and preparation steps that preserve class coverage and prevent leakage, and we defined an encoding and stratified splitting strategy to support reliable evaluation in the subsequent modeling chapter.

Chapter 5

Modeling and Evaluation

5.1 Introduction

This chapter covers the modeling and evaluation phase of the CRISP-DM methodology. Considering the critical nature of the domain and the complexity of the dataset, the training process is conducted as a series of controlled experiments, where results are carefully interpreted at each step to guide subsequent decisions. We begin with a state of the art review before presenting and justifying our model choice, then detail the experimental process from the initial baseline through iterative experiments, hyperparameter tuning, threshold optimization, and final model evaluation.

5.2 State of the art

In the network intrusion detection domain, various approaches have been proposed and studied. In this section, we review the most prominent methods, comparing their principles, strengths, and limitations, before selecting the one best suited to our project's requirements and dataset characteristics.

Given the nature of our dataset, CIC-IDS2017, we narrowed our research focus to classical machine learning approaches. While deep learning is proven to be powerful in domains such as image and text processing, it is generally ill-suited for tabular data. Furthermore, its black-box nature makes interpretability and explainability difficult, a critical concern in a security context where understanding model decisions is essential. Deep learning also requires large amounts of data to generalize well, which makes it all the more unsuitable as our dataset contains minority classes with as few as 11 samples, reduced to only 9 in the training set after the train-test split. Furthermore, studies on CIC-IDS2017 have shown that tree-based models generally outperform deep learning approaches on flow-based tabular data [23].

Among classical machine learning algorithms, the most prominent ones are K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and gradient boosting.

- **K-Nearest Neighbors (KNN):** It is a lazy learning algorithm that requires no training phase, as it simply memorizes the training dataset rather than building an explicit model. It classifies new samples, at inference time, by computing its distance to all existing data points and assigning the majority class among its k closest neighbors in the feature space [24].

- **Decision Tree:** It learns by recursively splitting data, picking at each node the feature and threshold that best separates the classes, producing leaf nodes that represent class labels [25].

- **Random Forest:** An ensemble of parallel decision trees, where each tree is trained on a random subset of data using a random subset of features [26].

- **Gradient Boosting:** An ensemble of sequential decision trees, where each new tree is trained to correct the remaining errors of the previous one [27].

Table 5.1 provides a comparative overview of those 4 methods.

Table 5.1 – Comparison of machine learning methods

Criteria	**KNN**	**Decision Tree**	**Random Forest**	**Gradient Boosting**
Data	Low-dimensional	Tabular, mixed	Tabular, high-dim	Tabular, structured
Interpretability	Low	High	Medium	Low–Medium
Overfitting Risk	Medium	High	Low	Low (with tuning)
Performance	Moderate	Moderate	Good	Best
Training Speed	None (lazy)	Fast	Medium	Slow
Inference Time	Slow	Very fast	Fast	Fast
Class Imbalance	Poor	Moderate	Good	Good (with configuration)
Feature Importance	No	Yes	Yes	Yes
Hyperparameter Sensitivity	Low (only k)	Low	Medium	High

KNN, while simple and capable of capturing complex patterns, suffers from slow inference time as it scans the entire dataset [28], making it impractical in network intrusion detection when time is of the essence. Decision Tree is fast and interpretable but prone to overfitting [29], a critical concern for this domain. Random Forest handles overfitting better, with medium training speed, fast inference time, and good performance [30], making it a strong candidate. Gradient Boosting, on the other hand, offers the best performance and handles class imbalance well [31], both critical for our dataset. Its inference time is fast despite its slow training speed, and while it requires careful hyperparameter tuning, its overfitting risk remains low when properly configured, and its sequential error correction is helpful for minority classes.

5.3 Adopted approach

We select Gradient Boosting as the modeling approach for this project, prioritizing performance, inference speed, robustness to overfitting, and suitability for tabular and imbalanced data. We choose XGBoost (extreme Gradient Boosting) for its efficiency.

XGBoost builds trees sequentially, where each boosting round corrects the errors of the previous one using both the Gradient and the Hessian of the loss function.

- **Gradient:** Indicates how wrong the previous prediction was and in which direction to correct it.
- **Hessian:** Measures the curvature of the error, making the corrections more precise and confident.

In our case, multiclass classification, a separate tree is built for each class at every boosting round, allowing the model to refine each class prediction independently. A node is only split when the gain is significant enough, preventing unnecessary complexity. All trees then work together, each contributing a weighted correction, to minimize the overall error. Additionally, XGBoost incorporates L1 and L2 regularization to further control overfitting, and exploits parallelism during tree construction, making it significantly faster than traditional gradient boosting despite its sequential nature.

5.4 Training and optimization

This section details the full training and optimization process of the XGBoost classifier on the CIC-IDS2017 dataset.

5.4.1 Initial setup

The initial setup covers three foundational decisions that govern all subsequent experiments: the evaluation metric, the validation strategy, and the initial model parameters.

Metric selection

The CIC-IDS2017 dataset is inherently imbalanced. `BENIGN` traffic constitutes roughly 80% of the dataset, reflecting the reality of network traffic where attacks are rare events. Within the attack classes themselves, representation varies significantly; some classes contain hundreds of thousands of samples while others hold fewer than a dozen. This makes metric selection a non-trivial decision that directly affects how honestly model performance is reported.

Accuracy is an intuitive metric but an unreliable one here. It counts correct predictions across all samples divided by the total.

It is defined as [32]:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

TP (True Positives)	: correctly detected positive samples
TN (True Negatives)	: correctly detected negative samples
FP (False Positives)	: negative samples incorrectly classified as positive
FN (False Negatives)	: positive samples incorrectly classified as negative

The sheer volume of `BENIGN` instances inflates the score regardless of attack detection quality. A model that predicts `BENIGN` for every single sample would still achieve over 99% accuracy while detecting zero attacks. This makes accuracy effectively blind to what actually matters.

Precision and Recall offer more granular insight into model behavior. Precision measures how many of the model's predictions for a given class are actually correct, and Recall measures how many of the true instances of that class the model successfully detected. They are defined as [32, 33]:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Both metrics are informative individually but insufficient alone; a model biased toward conservative predictions will report high precision at the cost of missing many attacks, while an overly permissive model will recover most attacks but introduce a high rate of false positives.

The F1-score addresses this by taking the harmonic mean of precision and recall, penalizing models that sacrifice one for the other. It is defined as [32]:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Standard F1-score or its weighted variant however, allows dominant classes to overshadow rare ones in the aggregate score, since each class's contribution to the final score is proportional to its sample count.

We therefore adopt the Macro F1-score as the primary evaluation metric, which computes F1-score separately for each class, then averages those scores, giving every class the same contribution regardless of how many samples it contains. It is defined as [34]:

$$\text{Macro F1} = \frac{1}{C} \sum_{i=1}^{C} \text{F1}_i$$

where C is the total number of classes and F1_i is the F1-score for class i.

This guarantees that every class, regardless of size, contributes equally to the final score. Per-class F1-scores are also tracked alongside it throughout all experiments to monitor individual class behavior.

Cross-validation

Given the challenges associated with the CIC-IDS2017 dataset, particularly the class imbalance issue and the limited number of samples in certain attack classes, we decide to adopt an experimental approach during the training phase and rely on evaluation metrics to guide our decisions for the final model configuration. This led us to use cross-validation.

Cross-validation is an evaluation technique that gives a more stable and reliable estimate of model performance. Instead of relying on a single train-test split, the dataset is divided into several folds, and the model is trained and evaluated multiple times, each time using a different fold as the validation set while the remaining folds are used for training. The final performance is then obtained by averaging the results across all runs. Several variants exist:

- Standard k-fold divides the data into k equal folds and rotates the validation fold across all k iterations. It is more practical, but it assigns samples to folds randomly, with no guarantee that rare classes appear in every fold. With classes as scarce as `HEARTBLEED`, a random split could place all its samples in the training folds and leave none for validation, or the reverse, making the evaluation meaningless for that class.

- Stratified k-fold addresses this directly. It applies the same k-fold rotation but enforces that each fold mirrors the class distribution of the full training set. If `HEARTBLEED` represents 0.0004% of the data, each fold will contain approximately that proportion. This guarantees that every class is present in both the training and validation portions of every fold, producing evaluation scores that are consistent and comparable across all k runs. We set $k = 5$. This choice is driven primarily by the scarcity of our rarest class: with only 9 `HEARTBLEED` samples in the training set, each fold sees approximately 2 instances. Increasing k would reduce this further, making per-fold evaluation for rare classes even less stable. Decreasing it would produce fewer evaluation rounds and increase variance in the aggregated score. Five folds represent a practical balance between evaluation stability and computational cost given this constraint.

Across all experiments, the reported performance corresponds to the mean Macro F1-score over the five folds, with per-class F1-scores tracked individually to monitor class-level behavior throughout.

Initial XGBoost parameters

The initial XGBoost model is trained with the following parameters:

- **objective:** `'multi:softprob'`; multiclass classification that provides probabilities for each class.

- **num_class:** `15`; the number of the target classes.

- **eval_metric:** `'mlogloss'`; the multiclass logarithmic loss used to evaluate the model during training.
- **tree_method:** `'hist'`; a histogram-based tree building algorithm that is very efficient for large datasets.
- **n_estimators:** `1000`; the maximum number of boosting rounds, set high to allow the model to fully converge.
- **max_depth:** `6`; the maximum depth of each tree, balancing between model complexity and generalization.
- **learning_rate:** `0.1`; the step size controlling how much each new tree contributes to the final model, allowing for steady learning without overshooting optimal solutions.
- **subsample:** `0.8`; the fraction of training samples used per tree, introducing randomness that helps prevent overfitting.
- **colsample_bytree:** `0.8`; the fraction of features used per tree, reducing overfitting by exposing each tree to a diverse feature subset.
- **min_child_weight:** `3`; the minimum sample weight required in a child node for a split to happen, preventing the model from creating overly specific splits on very few samples.
- **random_state:** `42`; ensures reproducibility across runs.
- **device:** `'cuda'`; enables GPU training for faster computation compared to CPU.
- **early_stopping_rounds:** `50`; training is stopped if the loss does not improve for 50 consecutive rounds, preventing overfitting and avoiding unnecessary computation.

5.4.2 Baseline training

A baseline model should be the first model built before moving toward more complex approaches, as it acts as a reference point for evaluating the performance of future configurations.

If a more advanced configuration fails to outperform the baseline or produces even lower results, this may indicate that the additional complexity provides little practical benefit. In such cases, the issue may not originate from the model itself, but may instead suggest potential limitations related to the dataset.

Furthermore, baseline results provide guidance for future steps. For example, if a classification model fails to predict certain classes, efforts should be placed on addressing this flaw.

For these reasons, we establish a baseline model by training XGBoost without applying any additional optimization techniques.

The baseline training results divide into expected and unexpected outcomes, as shown in Figure 5.1:

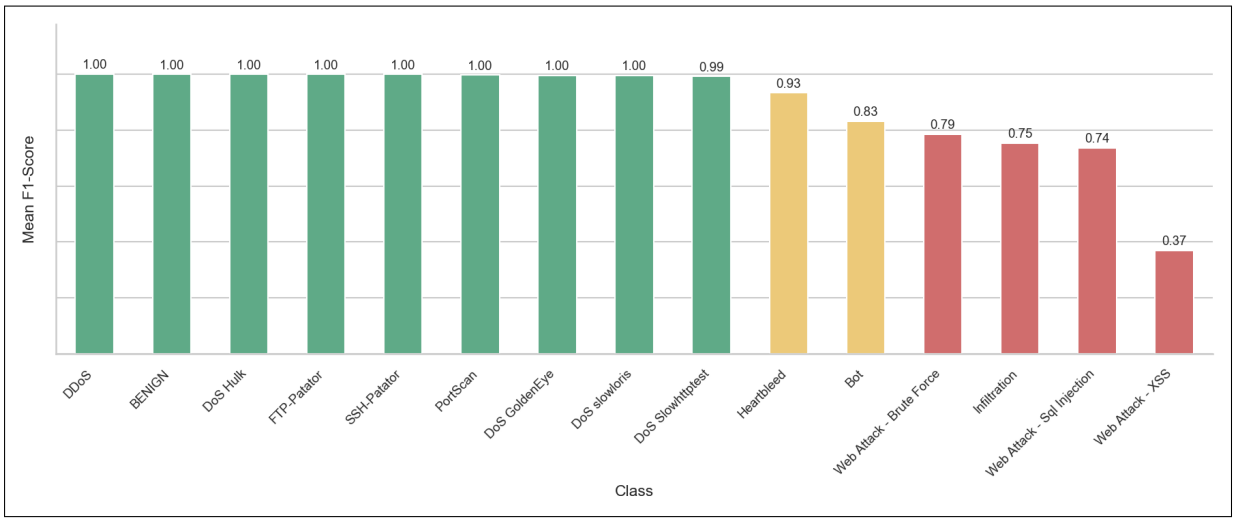


FIGURE 5.1 – Baseline F1-scores per class

Expected outcomes

Classes with large sample counts perform exactly as anticipated. Attack types such as DoS Hulk, PortScan, and DDoS, each with over 100,000 training samples, along with moderately represented classes like DoS GoldenEye, FTP-Patator, SSH-Patator, DoS Slowloris, and DoS Slowhttptest, each above 5,000 samples, achieve a high F1-score of 0.99 The model has enough examples to learn stable, discriminative boundaries for these classes.

At the other end of the spectrum, Heartbleed and Infiltration achieve comparatively lower performance, with F1-scores of 0.93 and 0.75 respectively. This behavior is expected, as only 9 Heartbleed samples and 29 Infiltration samples remained after preprocessing, which makes learning more challenging for the model. Nevertheless, the results remain promising given the extremely limited training data, and further improvement could likely be achieved.

Unexpected outcomes

- **Web attacks:** The most notable results come from the Web attack subclasses. We anticipated that Web Attack - SQL Injection would struggle, given only 19 training samples. Unexpectedly, all three Web Attack subclasses perform poorly: Brute Force achieves an F1-score of 0.79, SQL Injection 0.74, and XSS, the lowest-performing class, an F1-score of 0.37, despite having 522 training samples, which is typically sufficient for a tree-based model to learn from.

The confusion matrix is computed to identify the specific misclassification patterns. As shown in Figure 5.2, there is a consistent mutual confusion between XSS and Brute Force. XSS is misclassified 345 times as `Web Attack - Brute Force`, and `Web Attack - Brute Force` is misclassified as XSS 167 times. Both are also occasionally classified as `BENIGN`. This indicates that the model fails to establish meaningful separability between these categories in the feature space.

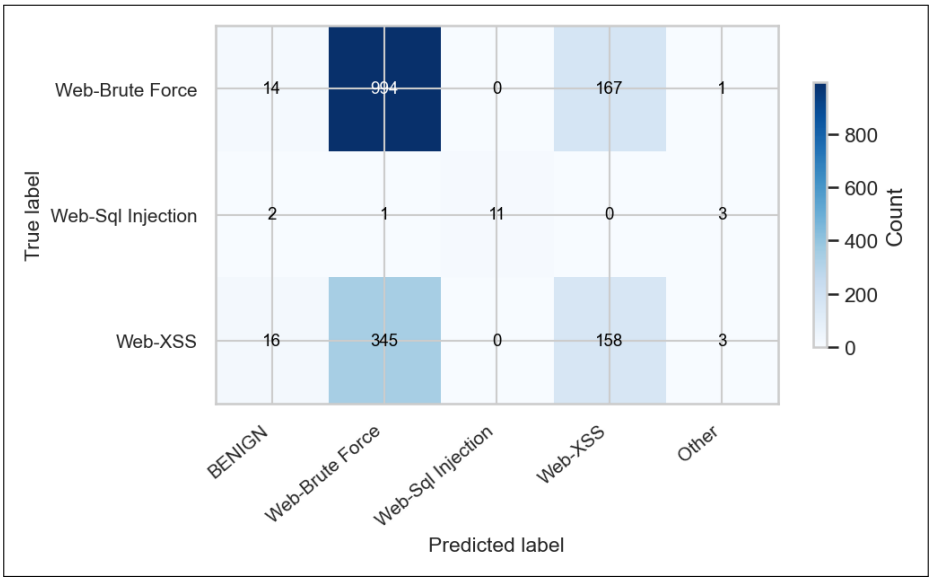


Figure 5.2 – Web attacks baseline confusion matrix

Although the Web Attack subclasses remain relatively underrepresented compared to dominant classes in the dataset, their consistently poor performance suggests something potentially more fundamental than class imbalance or sample scarcity alone.

Web attacks are payload-based attacks. An XSS carries a malicious script in an HTTP request body, and an SQL injection crafts SQL commands by tampering with input fields, interfering with database queries. Flow-based datasets like CIC-IDS2017 record features derived from packet timing, length distributions, and header-level metadata; there is no payload field. These features are evidently insufficient to capture the semantic structure of such attacks.

- **`Bot` attack:** `Bot` is still relatively underrepresented compared to dominant classes; however, with approximately 1,500 training samples, it was initially expected to provide sufficient information for the model to learn meaningful patterns. Despite this, its performance remains average, with an F1-score of 0.83.

  Considering the attack's nature might explain why the model is failing it. A `Bot` attack typically operates in multiple phases. The first is the infection phase, often done through phishing or malicious download. It occurs without any visible indications, and the user remains unaware that their machine is under the control of an attacker, effectively turning it into a "zombie" machine.

The second is Command-and-Control (C2) communication where the infected machine establishes contact with an external server controlled by the attacker to receive instructions and periodically report its status. Finally, the botnet is used to launch attacks such as `DDoS` attacks, spam distribution, or data exfiltration. CIC-IDS2017 represents the second phase.

The confusion matrix in Figure 5.3 shows that the bot attack is misclassified 351 times as `BENIGN`. This is due to the fact that, in this phase, the attacker intentionally mimics benign traffic patterns in order to remain undetected, making it difficult for the model to distinguish between malicious and normal behavior. Moreover, since the dataset represents each flow independently, it fails to capture the temporal nature of bot behavior, particularly periodic communication patterns across sessions.

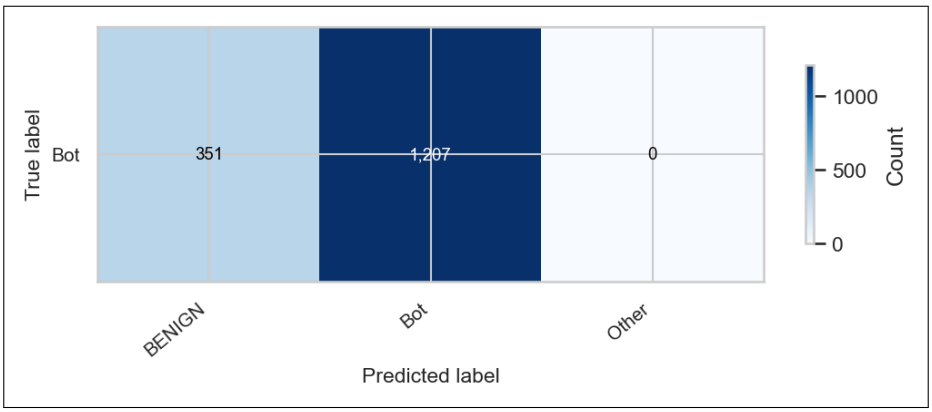


FIGURE 5.3 – Bot attack baseline confusion matrix

The baseline results show that underperforming classes do not fail for a single unified reason. Instead, two different patterns emerge. On one hand, classes such as `HEARTBLEED` and `INFILTRATION` appear to be primarily affected by extreme sample scarcity. On the other hand, `BOT` and Web Attack subclasses exhibit behavior that suggests additional limitations related to the nature of the data representation itself.

Despite this distinction, all these classes are still, to varying degrees, underrepresented compared to dominant classes. For this reason, it remains reasonable to investigate whether standard imbalance mitigation techniques can improve their performance before drawing definitive conclusions about the nature of these limitations.

5.4.3 Iterative experiments

The class imbalance problem can be addressed at two levels with either an algorithm-level approach or a data-level approach:

- **Algorithm-level: Class weights**

  XGBoost is a gradient boosting model that learns by minimizing a loss function. For every wrongly predicted sample, it multiplies the loss associated with it by its assigned weight, which is one by default, meaning the sample count is what primarily drives the loss function.

In highly imbalanced datasets such as CIC-IDS2017, the model naturally optimizes for the majority class because of its higher sample count.

In XGBoost, sample weights are used to adjust the importance of each individual training instance in the loss function, effectively penalizing the model more heavily for misclassifying samples. By assigning higher weights to the minority class, this forces the algorithm to focus on correctly predicting those rare instances in subsequent boosting rounds.

This technique is a standard way to handle imbalanced datasets without modifying the training data.

- **Data-level: Data augmentation**

  SMOTE (Synthetic Minority Oversampling Technique) generates new synthetic samples rather than duplicating existing ones. SMOTE works by identifying the k-nearest neighbors of an instance, then selects one at random. It then generates a new synthetic point using linear interpolation; it computes the distance between the two points, multiplies the difference by a random number between 0 and 1, and adds this value to the original sample, creating the artificial new sample.

  Although SMOTE becomes risky when applied to classes with very low sample counts, as the synthetic samples are created from a very limited neighborhood, which reduces diversity and can lead to unrealistic decision boundaries. This may result in generating synthetic points that do not accurately represent the real characteristics of that class, and can introduce noise.

  In this case, random oversampling (ROS) becomes a better resort. It works by duplicating existing samples. It introduces no new information but ensures the class appears more frequently during training.

That said, we approach SMOTE with deliberate caution. Network intrusion data is sensitive. A synthetic sample generated by interpolating between two real flow records is not guaranteed to faithfully capture the attack pattern, which presents a real risk. It may introduce noise that disrupts the model's learned decision boundaries rather than reinforcing them. Given the critical nature of network intrusion detection, we favor XGBoost's class weighting over data augmentation, as it offers a simpler and safer mechanism that achieves a similar effect without modifying the training data. Nonetheless, we still run sampling experiments to evaluate whether the data-level approach provides any measurable gain.

Rebalancing experiments

We define a fixed set of experiments. If a new configuration involves altering the training set, the same experiments are rerun to ensure consistent evaluation:

1. Class weights
2. Sampling (SMOTE + ROS)
3. SMOTE and weights

Following baseline training, we attempt to improve the performance of weaker-performing classes, namely the Web Attack subclasses, HEARTBLEED, INFILTRATION, and BOT, through these series of experiments while monitoring the evaluation metrics.

Class weights

Under class weighting (balanced), INFILTRATION improves to an F1-score of 0.84 and HEARTBLEED reaches 0.96. However, BOT performance deteriorates from 0.83 to 0.79, while Web Attack classes show no measurable improvement.

A second weighting strategy is explored using two custom formulations. The first assigns weights inversely proportional to the square of the baseline F1-score, amplifying attention on the worst-performing classes disproportionately more. The second is a hybrid formula that combines a frequency-based term, the square root of the inverse class frequency, with an F1-based penalty, the inverse of the baseline F1-score, so classes that are both rare and poorly predicted receive the highest combined weight. Both formulas are normalized by their mean to keep training numerically stable.

Neither, however, outperforms the standard balanced weighting; no improvement is observed for the target classes, and this configuration is dropped from all subsequent experiment sets.

Sampling (SMOTE + ROS)

Sampling is applied only within the training folds during cross-validation. The resampling is fit exclusively on each training fold, never on the validation fold, to prevent any form of data leakage from the synthetic samples into the evaluation.

In SMOTE, the parameter k defines the number of nearest neighbors used to generate synthetic samples. We set k = 5, the default value, as it ensures a stable neighborhood structure for synthetic sample generation. This choice directly influences the minimum viable class size required for SMOTE to operate correctly.

Based on this constraint, we apply SMOTE only to classes with more than 50 training samples and ROS on the ones below. This threshold is not arbitrary; SMOTE requires at least $k + 1$ samples in the minority class to identify valid neighbors. Below that count, the neighborhood becomes degenerate; the algorithm would interpolate between the same few points repeatedly, producing synthetic data that is essentially noise rather than new information.

The resampling targets are set as follows: SMOTE is used to increase BOT samples from 390 to 3,000, WEB ATTACK - BRUTE FORCE from 294 to 3,000, and WEB ATTACK - XSS from 522 to 2,000. Random oversampling is applied to WEB ATTACK - SQL INJECTION from 17 to 300, HEARTBLEED from 9 to 300, and INFILTRATION from 29 to 300.

Under this configuration, Web Attack classes show no improvement. While INFILTRATION drops slightly compared to the weights configuration with F1-score = 0.82, HEARTBLEED reaches a perfect 1.0 score.

Sampling and weights

When combining sampling with class weighting, Bot performance drops further, whereas Infiltration recovers to an F1-score of 0.87 Web Attack classes remain unchanged, while Heartbleed maintains a perfect F1-score of 1.0.

The consistent finding across all four configurations is that Web Attack performance does not respond to any rebalancing approach. This confirms that this is not a class imbalance problem; it is a feature representation problem.

Relative to the baseline, Bot performance consistently drops under these configurations, which confirms that this is a dataset limitation.

Web attack class merging

As established, no combination of weighting or sampling improves Web Attack performance. The confusion matrix shows persistent mutual misclassification between XSS and Brute Force across every experiment. Their flow-level signatures are too similar to separate with the available features.

We therefore decide to merge the three Web Attack subclasses, Brute Force, SQL Injection, and XSS, into a single unified Web Attack class.

Because this modifies the training set, all four experimental configurations are re-run from scratch.

The merged Web Attack class immediately achieves F1-score = 0.98. However, Heartbleed drops from 0.93 to 0.86 relative to the original baseline, while Bot remains stable at 0.83.

When class weights are introduced, Heartbleed reaches an F1-score = 0.96 Infiltration recovers to 0.87, matching its performance before merging under the sampling and class-weighting strategy. Web Attack holds at 0.98.

Under sampling alone, Infiltration reaches only 0.80, Heartbleed drops to 0.93 Web Attack stays at 0.98.

Finally, combining sampling with class weights, Web Attack remains at 0.98 However, Infiltration drops to 0.79. Heartbleed reaches 1.0 F1-score again.

Across all four configurations, Web Attack consistently sits at approximately 0.98 regardless of whether weights or sampling are added. This confirms that merging alone is the decisive intervention; no additional rebalancing is necessary.

Heartbleed ablation

Heartbleed warrants a closer look. With only 9 training samples and 2 retained in the test set, cross-validation folds see approximately 2 Heartbleed samples per validation split. At that scale, the F1-score can take only a handful of discrete values, 0 when both are wrong, 0.67 in case only one is wrong, and a perfect 1.0 score means the model simply predicts both samples correctly.

More importantly, all resampling and rebalancing techniques are applied exclusively to the training folds and therefore do not affect the test distribution. As a result, regardless of whether weights or sampling are used, the final evaluation is always based on the same two test samples. This makes it hard to meaningfully assess the impact of these strategies on HEARTBLEED's generalization performance.

This initially motivates an ablation experiment in which HEARTBLEED is temporarily removed to assess its impact on the model's behavior.

Following this change, the complete set of experiments is rerun. However, the results are unexpected: INFILTRATION, which previously showed significant improvement under the merged WEB ATTACK configuration with class weighting and HEARTBLEED included, reaching its best F1-score of 0.87, drops once HEARTBLEED is removed while keeping the same merge and weighting setup.

As shown in Figure 5.4, removing HEARTBLEED under class weighting leads to a substantial decrease in INFILTRATION F1-score.

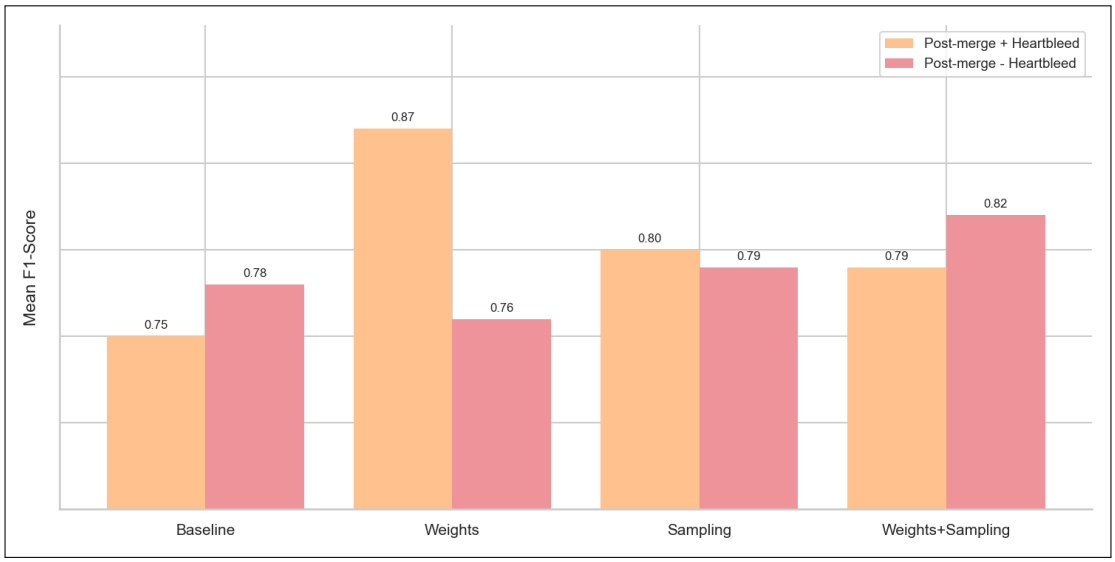


FIGURE 5.4 – Infiltration F1-score comparison across configurations

This behavior can be explained by the optimization mechanism of XGBoost. Under class weighting, HEARTBLEED receives a gradient scaling factor of approximately 19,000× relative to the BENIGN class. Each of its 9 training samples exerts as much optimization pressure on the model as a substantial fraction of the majority class flows. The model, responding to this extreme gradient signal, prioritizes correctly resolving the HEARTBLEED decision boundary.

This extreme weighting has an indirect structural effect on INFILTRATION. In XGBoost, each tree is built by recursively partitioning the feature space, and each split is chosen to minimize the weighted gradient loss across all classes simultaneously. When HEARTBLEED receives 19,000× weight, its samples impose disproportionate pressure on how certain boundaries are drawn in the shared feature subspace.

Some of these boundaries, though oriented toward separating HEARTBLEED from BENIGN, incidentally create a favorable partition for INFILTRATION samples as well, because the two classes occupy adjacent regions of feature space, the cut drawn to isolate HEARTBLEED simultaneously pulls INFILTRATION away from BENIGN. Removing HEARTBLEED under weighting relaxes that boundary, and INFILTRATION loses the incidental separation from BENIGN.

To conclude, HEARTBLEED and INFILTRATION have no structural relationship in the data. The dependency is induced entirely by the extreme weight assigned to HEARTBLEED.

The ablation results demonstrate that removing or amplifying HEARTBLEED affects specific aspects of the model's behavior; under class weighting, its strong gradient signal contributes to sharpening nearby decision boundaries in the feature space, which indirectly benefits INFILTRATION performance.

This observation led to a reassessment of the initial interpretation of the metric limitation.

While the extremely small number of test samples prevents statistically robust metric estimation, it does not imply that the model fails to learn the class during training. The training process still incorporates HEARTBLEED samples, allowing the model to adjust its internal decision boundaries accordingly.

More importantly, despite the severe data scarcity, both HEARTBLEED and INFILTRATION still achieve strong performance, reaching F1-scores of 0.93 and 0.87 respectively under the selected configuration. In the context of network intrusion detection, retaining additional attack coverage presents no practical downside when the model maintains acceptable predictive performance.

In light of the ablation findings, we decide to retain HEARTBLEED.

5.4.4 Final model configuration

The final model configuration carried forward into hyperparameter tuning consists of merging the Web Attack subclasses into a single class, retaining HEARTBLEED, and applying balanced class weights. This configuration achieves a Macro F1-score of 0.97.

At the class level, nine classes achieve near-perfect performance with F1-scores of approximately 0.99. HEARTBLEED reaches an F1-score of 0.96, INFILTRATION achieves 0.87, and the merged WEB ATTACK class obtains an F1-score of 0.98.

5.4.5 Hyperparameter tuning

Following the final model configuration, we proceed to hyperparameter tuning, defining a search space over the most impactful parameters and applying a search strategy to find the combination that maximizes model performance.

Search space

The parameters selected for tuning are as follows:

- **max_depth:** Controls the complexity of each tree. A high value risks overfitting, and a low value risks underfitting, making it critical for generalization.
- **min_child_weight:** Controls the node splits. High values lead to fewer splits, potentially making it harder for the model to learn minority-class patterns; low values allow more splits at the risk of overfitting.
- **subsample and colsample_bytree:** They respectively control the subset of samples and features each tree is trained on. Finding the right values introduces just the right randomness to improve generalization without losing too much training signal.
- **reg_alpha and reg_lambda:** L1 and L2 regularization with L1 pushing less useful weights toward zero while L2 shrinks all weights to stabilize predictions. High regularization makes the model too simple to detect subtle attack patterns and a lower one risks overfitting to majority class noise.
- **learning_rate:** Controls the learning rate of each boosting round. High value leads to faster convergence but can overshoot the optimal solution, low value leads to slower convergence but more stable learning.

Each parameter is assigned a range of values to explore, as shown in Table 5.2.

Table 5.2 – Search space

Parameter	Type	Range
max_depth	Integer	[4, 8]
min_child_weight	Integer	[1, 20]
subsample	Float	[0.7, 0.95]
colsample_bytree	Float	[0.7, 0.95]
reg_alpha	Float	[0, 2.0]
reg_lambda	Float	[0.5, 5.0]
learning_rate	Float	[0.03, 0.15]

Search method

Having defined the search space, we proceed to decide the search method. Three search strategies are considered:

- **Grid search:** Tries every possible combination of the search space, making it expensive; with 7 parameters and continuous ranges, even a coarse discretization yields thousands of combinations, which costs too much time and computational resources.

- **Random search:** Tries a random fixed number of combinations, which is much cheaper than grid search and often yields good results faster; however, it is not guaranteed to cover everything. It treats each parameter independently and has no memory, which may lead to wasting trials in regions of the search space already known to be poor.

- **Bayesian optimization:** A smart search method where the results of previous trials are used to choose better next trials. It builds a probabilistic model of the objective function and uses it to select the most promising hyperparameters to evaluate in the true objective function.

Bayesian optimization with Optuna is selected, as it is a smart method yielding better results at a much lower computational cost. It uses a Tree-structured Parzen Estimator (TPE) that splits the trial history into two groups: one with the best-performing trials and another with the rest. It then proposes the candidate hyperparameters that maximize the ratio between the two distributions, focusing sampling on promising regions while avoiding areas already known to perform poorly. It can also capture parameter dependencies as it models each parameter's distribution conditionally.

Tuning results

After 100 trials, the best configuration is found at trial 91, achieving a Macro F1-score of 0.976 with an improvement of 0.006 over the untuned model. The INFILTRATION F1-score is improved from 0.87 to 0.91, and HEARTBLEED also has a significant improvement from 0.96 to 1.0, while the merged WEB ATTACK class remains stable at 0.98. The best hyperparameters are as follows:

- **max_depth:** 6
- **min_child_weight:** 1
- **subsample:** 0.86
- **colsample_bytree:** 0.93
- **reg_alpha:** 0.075
- **reg_lambda:** 1.02
- **learning_rate:** 0.1

Throughout the study, the F1-score is improving steadily beyond 50 trials before converging around trial 80, as shown in Figure 5.5. The blue dots scattered across the search space indicate that the search is still exploring promising regions.

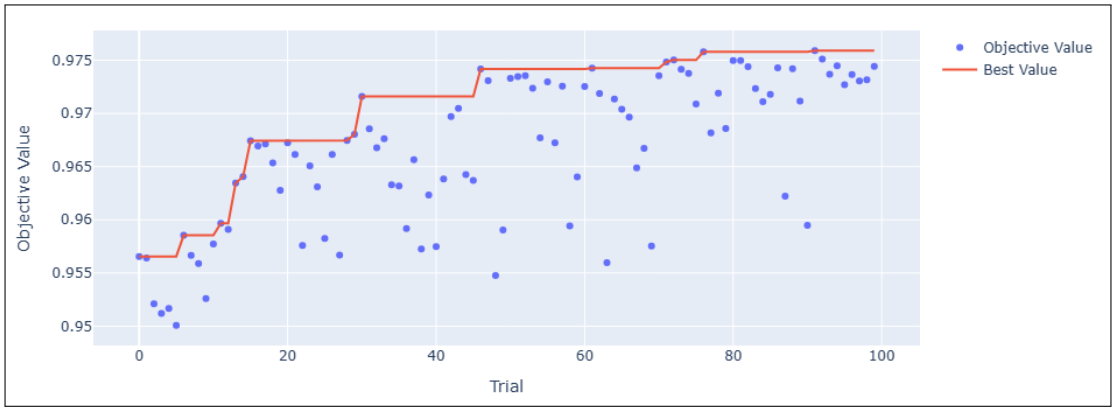


FIGURE 5.5 – Optimization history

5.4.6 Adding thresholds

Beyond class weighting and hyperparameter tuning, two additional threshold mechanisms are applied to further refine the model's decision-making.

Decision threshold

XGBoost assigns each sample to the class with the highest predicted probability. In an imbalanced dataset, this is problematic because the model learns a bias toward majority classes during training, which is expressed at prediction time through higher predicted probabilities for those classes, making it harder for minority classes to be predicted even when the model has a reasonable signal for them. To address this at the decision level, a threshold is optimized independently for each class on the out-of-fold predictions from cross-validation. By acting as a per-class scale that normalizes each class's probability, minority classes are given a fairer chance at being predicted without modifying the model itself.

Confidence threshold

A second limitation arises from the closed-world assumption of the model. The model is trained on a fixed set of known attack classes and will always assign a sample to one of them, even when encountering traffic that does not resemble any of them. In practice, new attack types emerge continuously, and the model has no mechanism to express uncertainty. When faced with an unknown pattern, it will most likely classify it as `BENIGN`, which is the worst possible outcome in a security context. To address this, a confidence threshold is set on the maximum probability across all classes. When the model's highest confidence for any class falls below this threshold, the flow is flagged as unknown rather than forced into a potentially wrong classification, leaving it for human review.

5.4.7 Final model training results

The final model is trained on the full training set using the tuned hyperparameters with balanced class weights. Since training on the full dataset leaves no validation set, early stopping cannot be applied. Instead, the number of estimators is fixed by averaging the best iteration across the five cross-validation folds, which gives a mean of 1086 trees. A 20% buffer is added on top to account for the fact that more training data generally requires more trees to converge, bringing the final number of estimators to 1300.

The model is evaluated on the held-out test set using the decision thresholds. The final model achieves a Macro F1 of 0.98, exceeding the cross-validation mean of 0.96, indicating that the model generalizes well and did not overfit to the training data.

At the class level, nine classes achieve near-perfect F1-scores above 0.99. `Heartbleed` and `Infiltration` both reach a perfect F1-score of 1.0 on the test set. This result is attributed to two factors: training on the full dataset exposes the model to all available patterns for these classes, and the test samples are similar enough to the training data

for the learned decision boundaries to classify them correctly. It should be noted, however, that with only 2 HEARTBLEED and 7 INFILTRATION test samples, this result is not statistically robust and should be interpreted with caution. A more reliable evaluation of these classes would require testing on additional data from independent sources, which is considered a future direction. The merged WEB ATTACK class achieves an F1-score of 0.98, while BOT reaches 0.84.

Figure 5.6 shows the final F1-scores for each class.

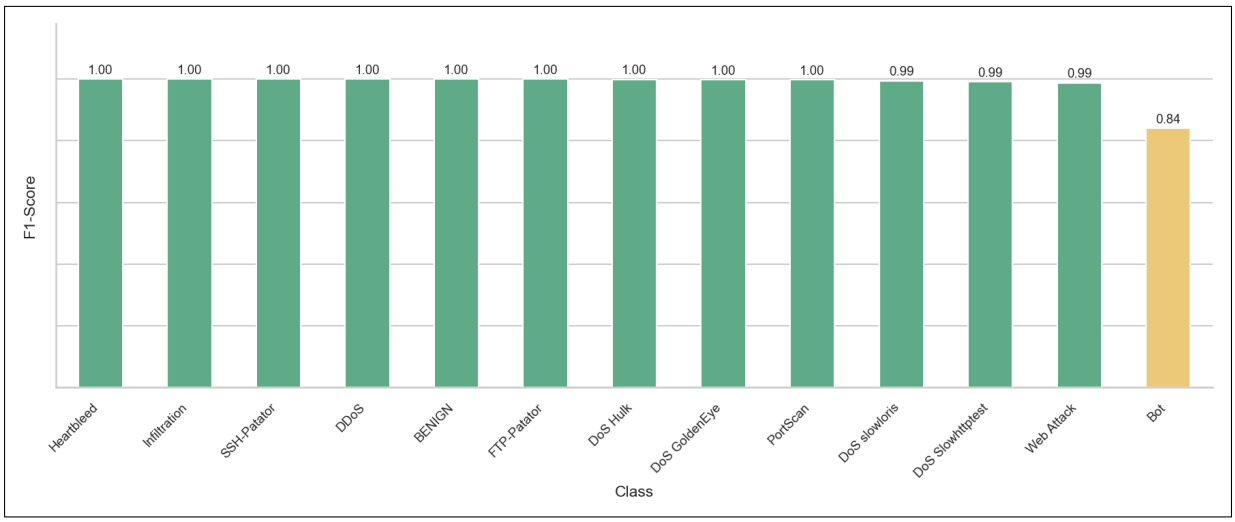


FIGURE 5.6 – Final F1-score per class

5.5 Conclusion

This chapter presented the full modeling and evaluation pipeline of the project. Through a series of controlled experiments, class imbalance was systematically addressed, leading to a final model configuration that achieves a Macro F1-score of 0.9852 on the held-out test set. The next chapter presents the realization of the system.

Chapter 6

Realization

6.1 Introduction

This chapter presents the realization phase of the system. Prior to this stage, the project went through several phases, starting with requirements specification and conceptual design, followed by the CRISP-DM phases of data understanding, data preparation, modeling, and evaluation. The final stage corresponds to the deployment phase of the CRISP-DM methodology.

We begin by presenting the development tools and technologies used in the implementation of the different system components. We then detail the model integration, which constitutes the core of the system by enabling the intrusion detection functionality within the web application. This is followed by an explanation of the deployment process, which defines the runtime environment in which the system operates. Finally, we present the system's results through the main user interfaces.

6.2 Development tools and technologies

This section presents the development tools and technologies used throughout the project. Table 6.1 summarizes the main technologies adopted, along with their respective roles within the system.

Table 6.1 – Development tools and technologies

Language	
	**Python**[35] Python is a high-level, interpreted programming language, widely used for scientific computing and rapid prototyping. In this project, it supports the data understanding, preparation, modeling, and evaluation scripts. It is also used to implement the inference service.

Machine Learning

Scikit-learn[36]

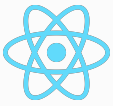
Scikit-learn is an open-source machine learning library that provides standardized APIs for preprocessing, model evaluation, and classical learning algorithms. It is used to support preprocessing operations, splitting strategies, and metric computation.

Backend

Spring Boot[37]

Spring Boot is an opinionated Java framework that accelerates the development of RESTful services through auto-configuration and production-ready defaults. It is used to implement the backend layer, manage persistence, and orchestrate communication between the web application and the inference API.

Frontend



React[38]

React is a JavaScript library for building interactive user interfaces using a component-based architecture and declarative state management. It is used to implement the frontend layer.

UI Styling

Tailwind CSS[39]

Tailwind CSS is a utility-first CSS framework that provides low-level classes to rapidly build consistent, responsive user interfaces. It is used to style and ensure a consistent design across the web application.

Database

MySQL[40]

MySQL is an open-source relational database management system (RDBMS) that implements the SQL data model. It is used to store persistent application data (e.g., network flows, alerts, and analysis results) in order to support querying and reporting.

Local Server

XAMPP[41]

XAMPP is a cross-platform local server distribution bundling common web stack components (e.g., Apache and database services) to simplify setup. It is used during prototyping to run the local stack quickly and validate database connectivity.

API Testing

Postman[42]

Postman is an API platform for constructing and executing HTTP requests, organizing test collections, and validating responses. It is used to test REST endpoints during development and to verify payload structures, responses, and error handling.

API

FastAPI[43]

FastAPI is a Python framework for building REST APIs. It is used to expose the detection model through a lightweight inference service.

Containerization

Docker[44]

Docker is a containerization platform. It is used to isolate system components and ensure consistent execution across environments.

Orchestration

Docker Compose[45]

Docker Compose is a tool for defining and running multi-container applications through a declarative configuration file. It is used to orchestrate the local multi-service stack and reproduce the platform setup using a single configuration.

Version Control

Git[46]

Git is a distributed version control system designed to track code changes and support branching and merging workflows. It is used to maintain project history and ensure traceability across development iterations.

Code hosting and collaboration

GitHub[47]

GitHub is a web platform for hosting Git repositories and supporting collaborative software development. It is used to host the project repository and manage collaboration through issues and pull requests.

IDE

Visual Studio Code[48]

Visual Studio Code is an extensible source-code editor providing language support, debugging capabilities, and integrated tooling through extensions. It is used as the main environment for Python scripts and frontend development.

IDE

IntelliJ IDEA[49]

IntelliJ IDEA is an integrated development environment for the Java ecosystem, with dedicated support for Spring-based projects and automated refactoring features. It is used for backend development, including dependency management and code refactoring.

6.3 Model integration

After training and validating the intrusion detection model, the next step consisted of integrating it into the application environment in order to enable prediction through the web platform.

FastAPI[43] was adopted as the communication layer between the model and the rest of the system. This approach ensures that the model is only loaded once during application startup, thereby reducing latency and improving prediction efficiency. In addition, FastAPI offers high-performance, lightweight architecture, asynchronous request handling, and automatic API documentation, making it well-suited for AI-based applications.

The implemented API exposes two main endpoints. The first endpoint, `/health`, is used to verify that the service is operational and the prediction model has been successfully loaded into memory, as illustrated in Figure 6.1. The second endpoint, `/predict`, receives a feature vector as input and returns the predicted attack class along with the associated confidence score and inference time, as shown in Figure 6.2.

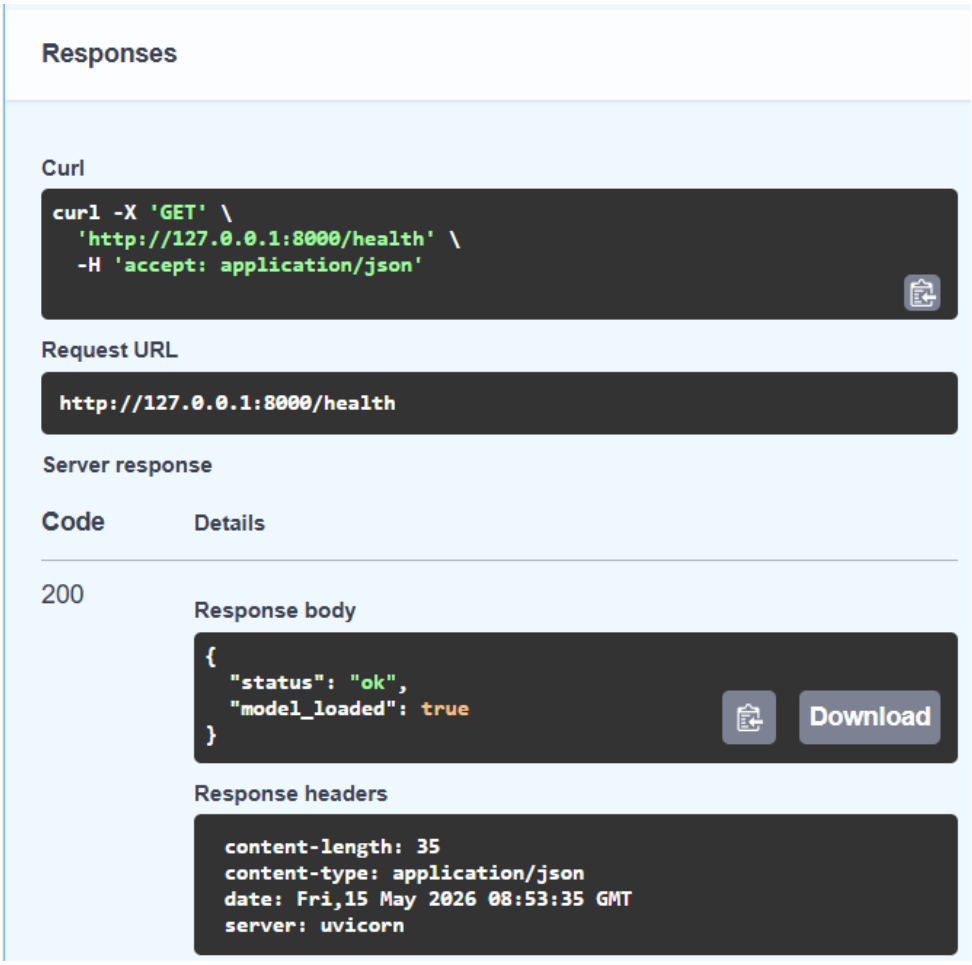


FIGURE 6.1 – FastAPI `/health` endpoint

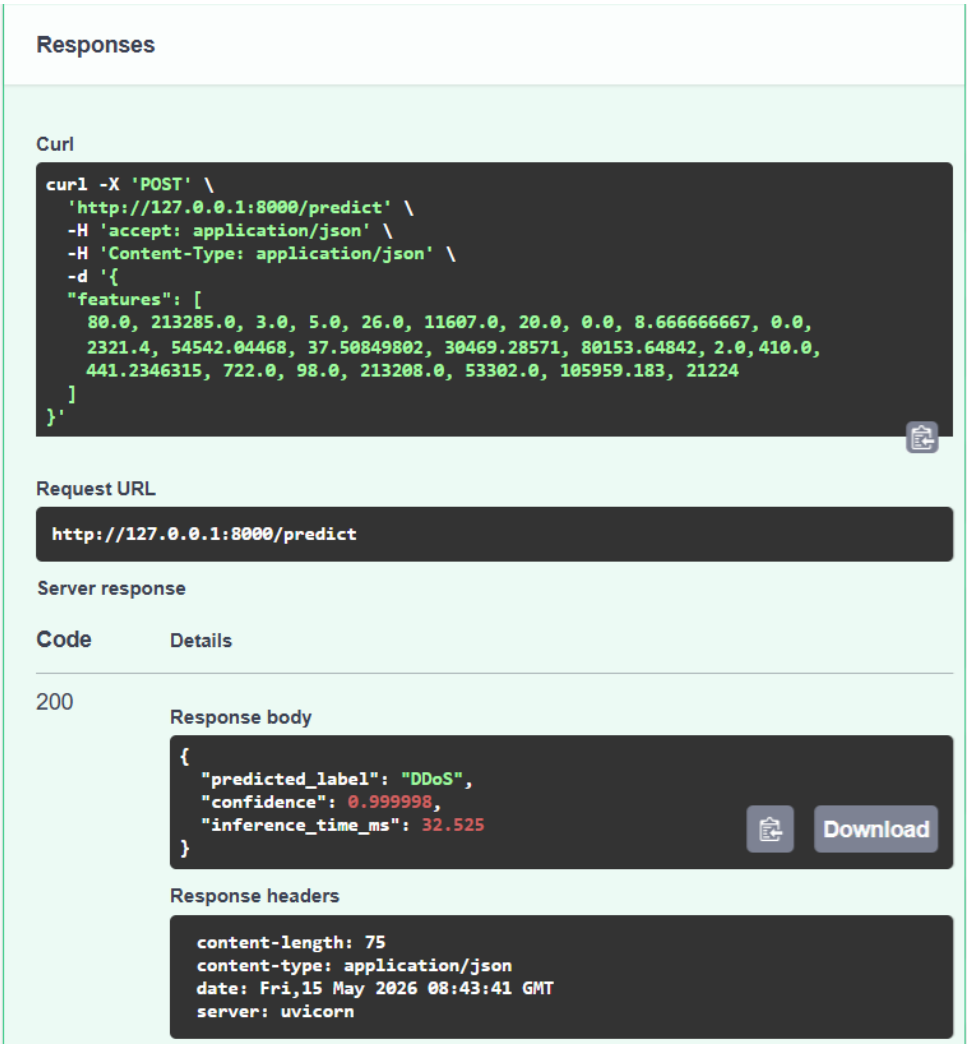


FIGURE 6.2 – FastAPI `/predict` endpoint

6.4 Deployment

Deployment refers to the runtime environment in which a software system is executed, including how its components are instantiated, organized, and made operational. It describes how the system moves from a development state to a working environment where all services interact and function together.

6.4.1 Containerization

Containerization is an approach that packages an application together with all its required dependencies, libraries, and configuration into a single isolated runtime environment called a container.

This approach ensures portability and consistency by allowing the application to run uniformly across different machines and operating systems. By isolating each service in a dedicated container, it also prevents dependency conflicts and improves system modularity.

Containerization was adopted to provide a controlled and reproducible execution environment for the different modules of the intrusion detection system.

The proposed system is decomposed into several independent services, each encapsulated within its own Docker container. The frontend is deployed as a React application, the backend is implemented using Spring Boot, the prediction model is exposed through a FastAPI container, and the database is managed using a MySQL container.

Each container is defined using a dedicated Dockerfile that specifies the environment configuration and dependencies required for execution. This modular structure improves system maintainability and ensures clear separation of concerns between the different components.

In addition, the Spring Boot service includes a persistent volume to support report generation functionality, allowing generated outputs to be stored beyond the lifecycle of individual containers.

6.4.2 Orchestration

To manage and coordinate the execution of the different containers, Docker Compose is used as an orchestration tool. It allows the system to be started as a single unified environment while maintaining the independence of each service.

Docker Compose defines the services, networks, and dependencies between containers, ensuring proper initialization order and communication between components. This orchestration layer simplifies deployment by eliminating the need to manually start each service individually and ensures that all parts of the system operate together coherently.

In practice, the complete stack can be launched using the command `docker-compose up -d`, which starts all containers in detached mode. Figure 6.3 shows an example of successfully launching the system containers using Docker Compose.

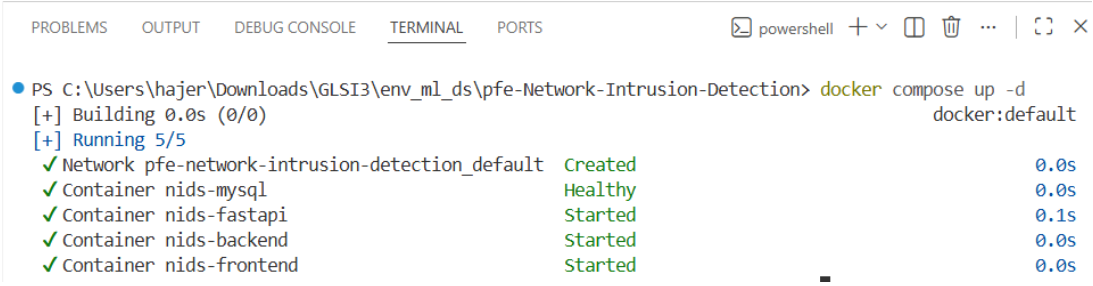


FIGURE 6.3 – Launching the system containers

6.4.3 Inter-Service communication

Communication between system components is based on HTTP requests within the Docker network. The frontend communicates with the backend through REST APIs, while the backend interacts with the FastAPI service to obtain prediction results from the trained model. The backend also handles data persistence by communicating with the MySQL database, and external notifications are sent through an SMTP service.

The system architecture is illustrated in the deployment diagram shown in Figure 6.4.

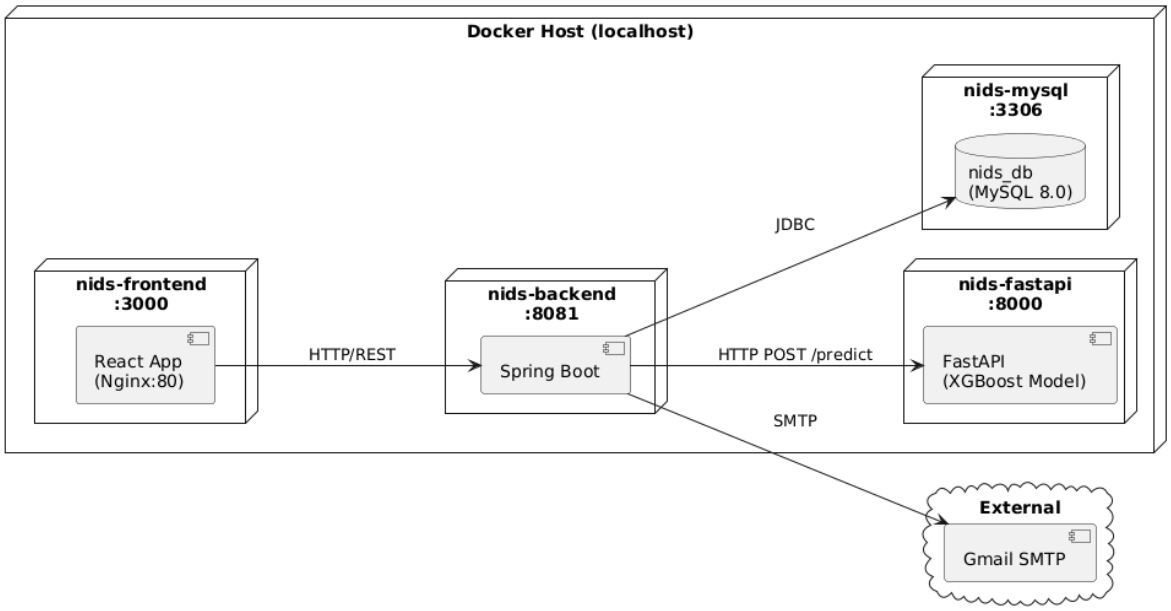


FIGURE 6.4 – Deployment diagram

6.5 User interfaces

This section presents the main user interfaces of the NIDS Web Application.

Use case «Authenticate» realization

The authentication interface allows the user to log in and access the platform features, as shown in Figure 6.5.

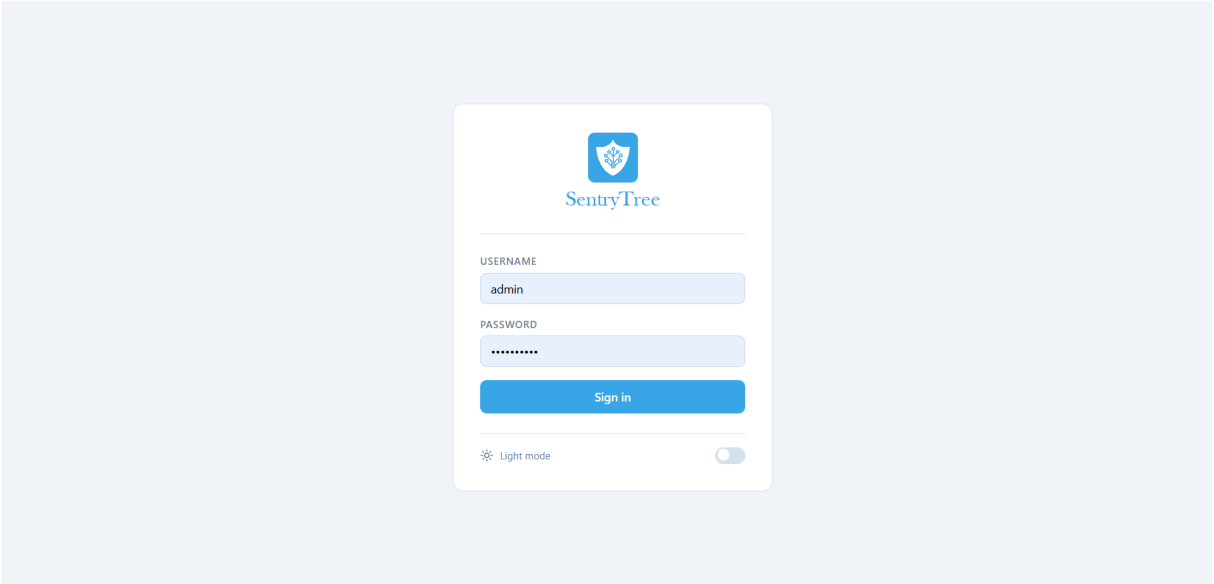


FIGURE 6.5 – Authentication interface

Use case «View Dashboard» realization

The dashboard interface provides a global overview of the system state and recent activity, as shown in Figure 6.6.

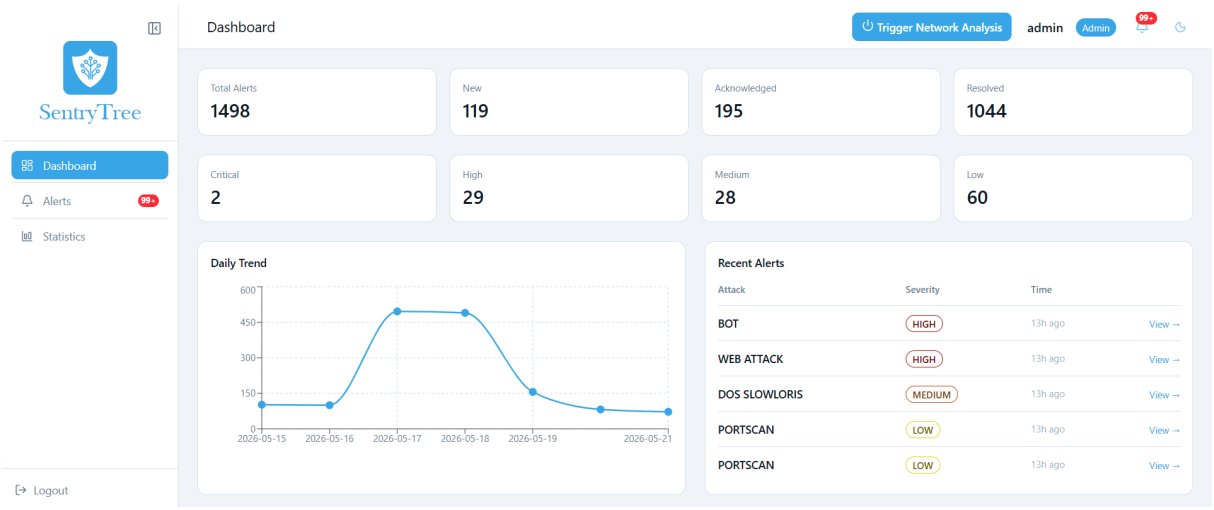


FIGURE 6.6 – Dashboard interface

Use case «View Statistics Summary» realization

The statistics summary interface provides aggregated metrics and trends for monitoring purposes, as shown in Figures 6.7 and 6.8.

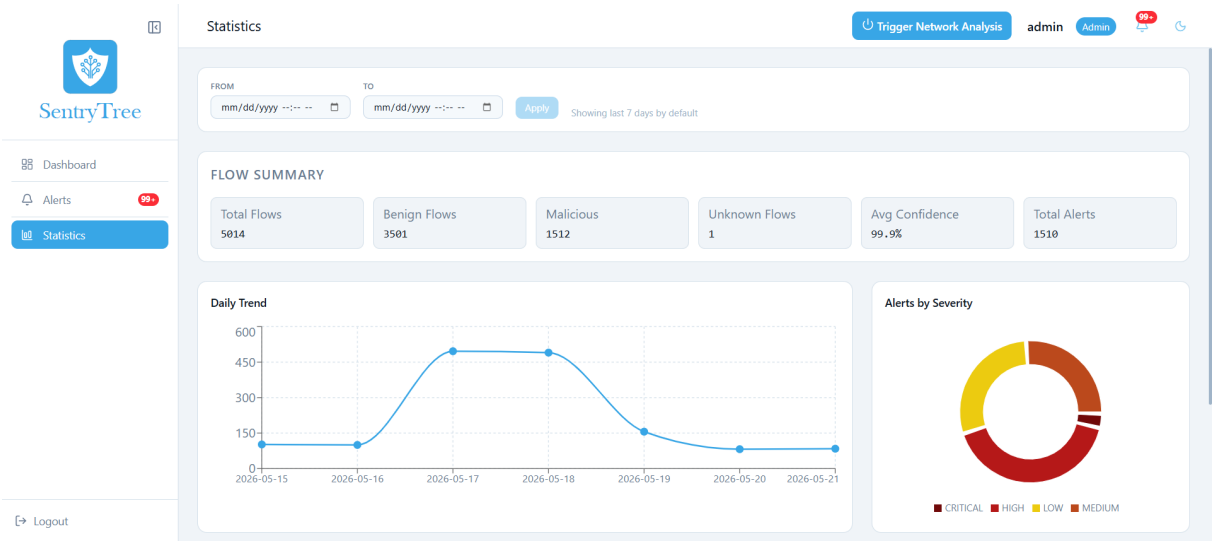


FIGURE 6.7 – Statistics summary interface 1

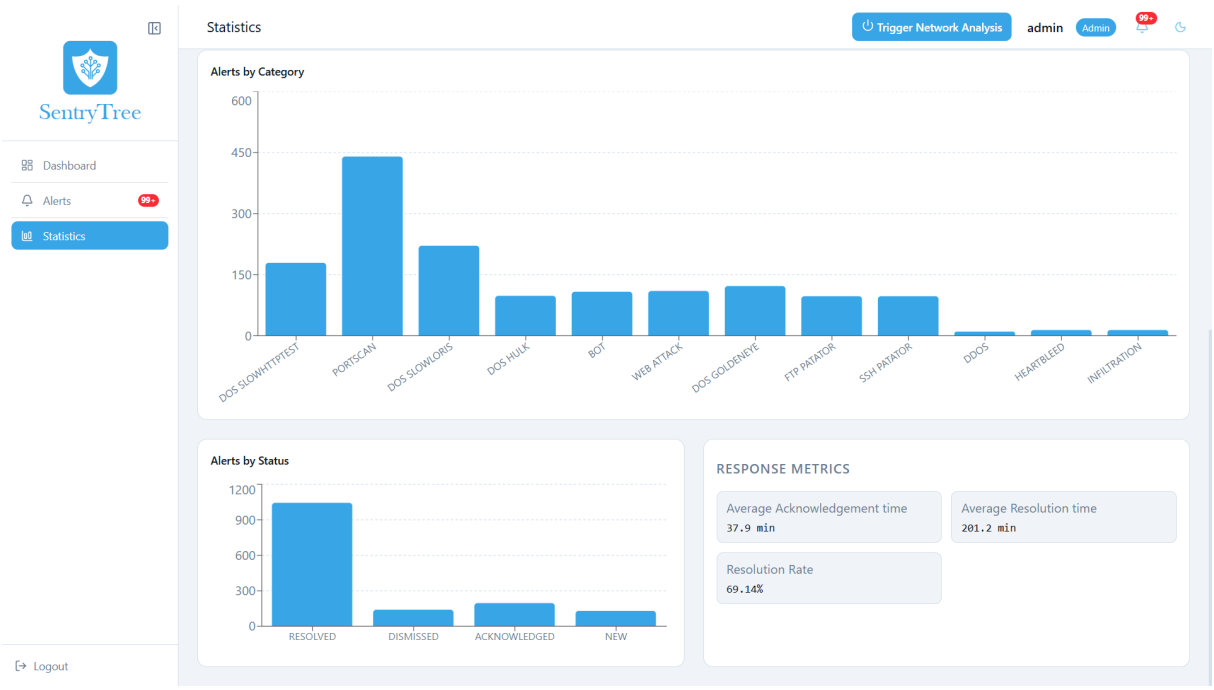


FIGURE 6.8 – Statistics summary interface 2

Use case «Trigger Network Analysis» realization

The analysis trigger interface allows an administrator to start a network analysis process, as shown in Figure 6.9.

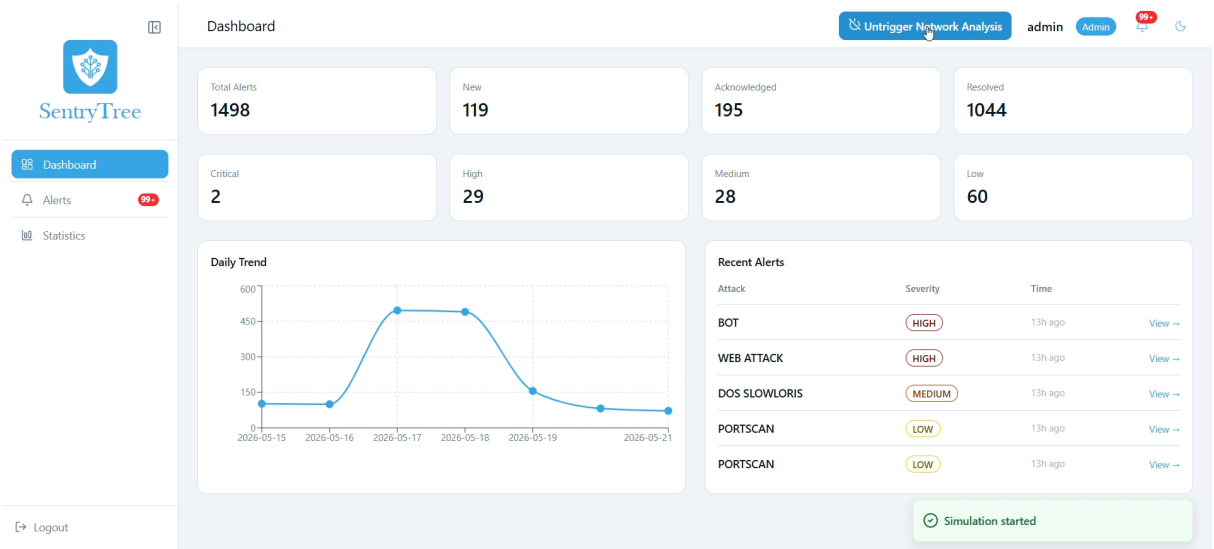


FIGURE 6.9 – Trigger network analysis interface

Use case «Send Alert» realization

The alert notification mechanism operates on two channels. First, when a new alert is generated, it is immediately pushed to the web application as an in-dashboard notification to highlight the event and indicate that new alerts are available, as shown in Figure 6.10. Second, email notifications are issued according to severity: a *critical* alert triggers an immediate email notification, while a *high* alert triggers an escalation email when it remains unacknowledged for more than 5 minutes, as illustrated in Figure 6.11.

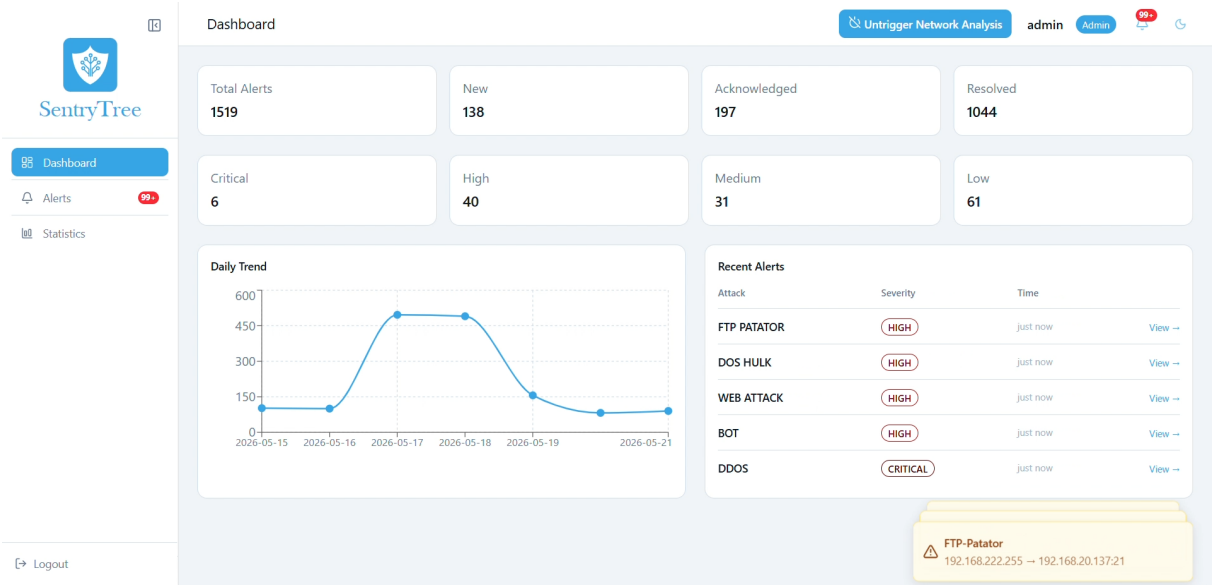


FIGURE 6.10 – In-dashboard alert notification

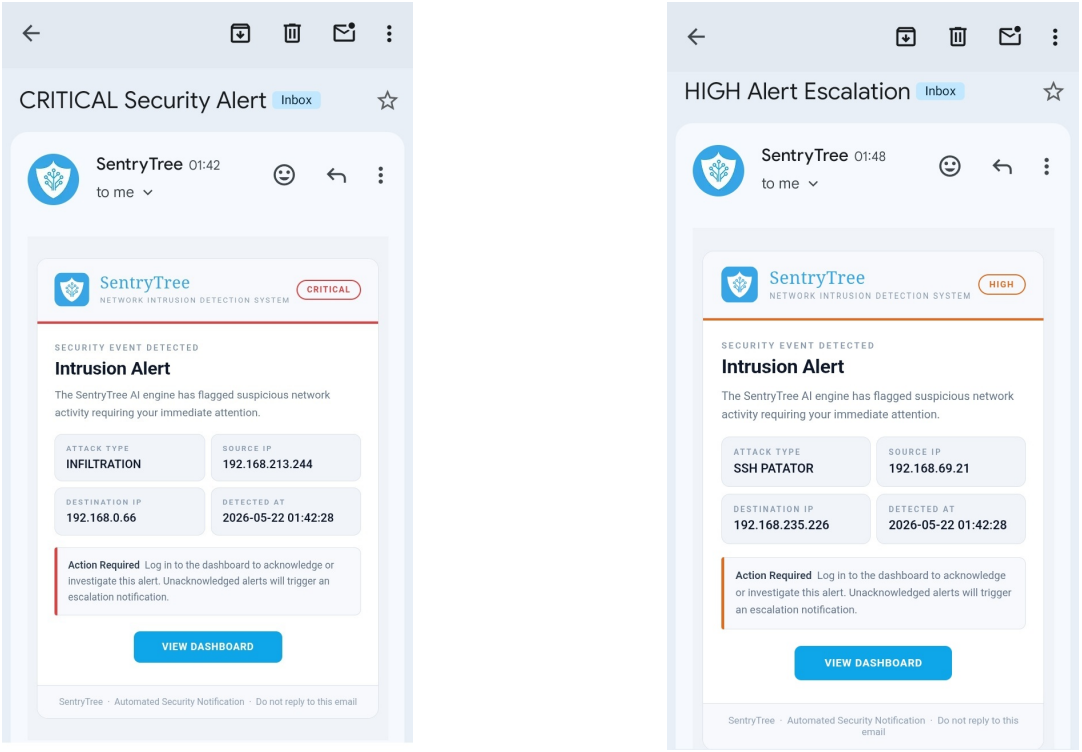


FIGURE 6.11 – Email notifications

Use case «View Alerts List» realization

The alerts list interface displays all detected alerts with key information for quick review, as shown in Figure 6.12.

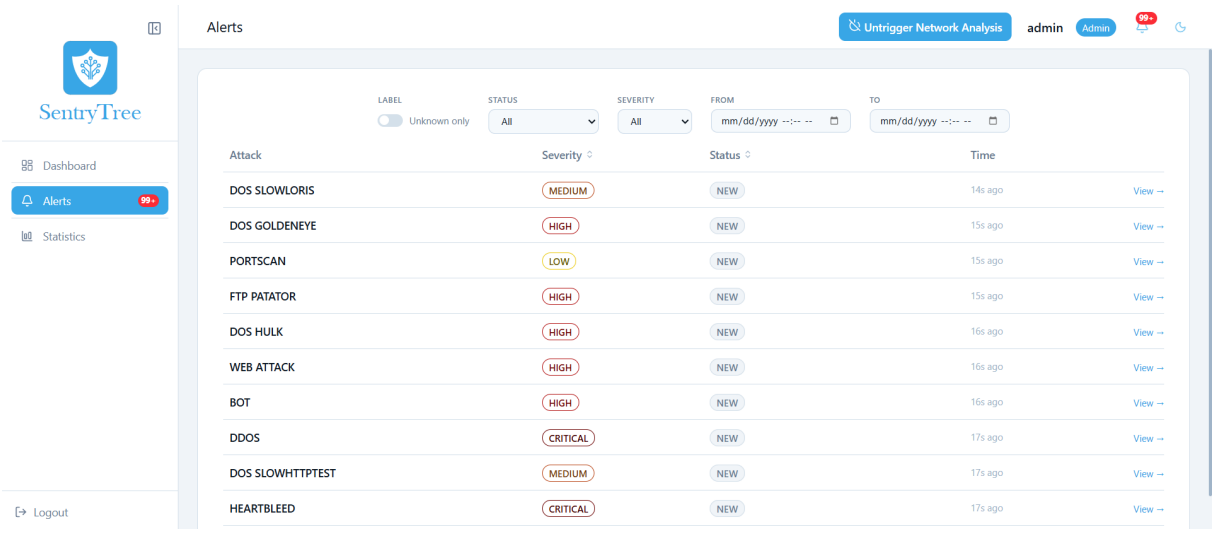


FIGURE 6.12 – Alerts list interface

Use case «Search Alerts» realization

The search interface enables filtering and searching through alerts by different criteria, as shown in Figure 6.13.

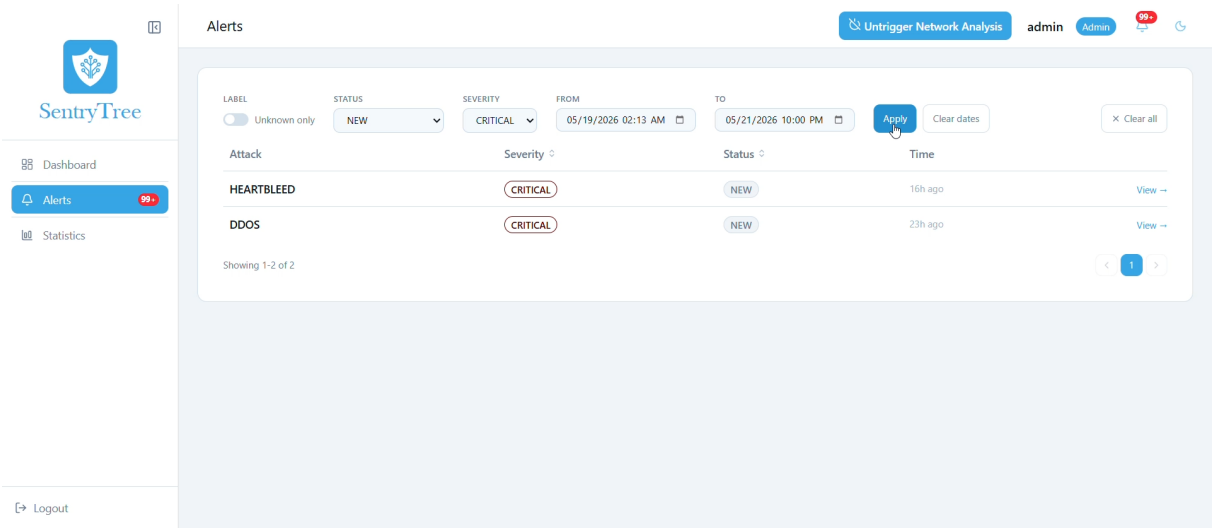


FIGURE 6.13 – Search alerts interface

Use case «View Alert Details» realization

The alert detail interface presents the full information of a selected alert, enabling deeper investigation, as shown in Figure 6.14.

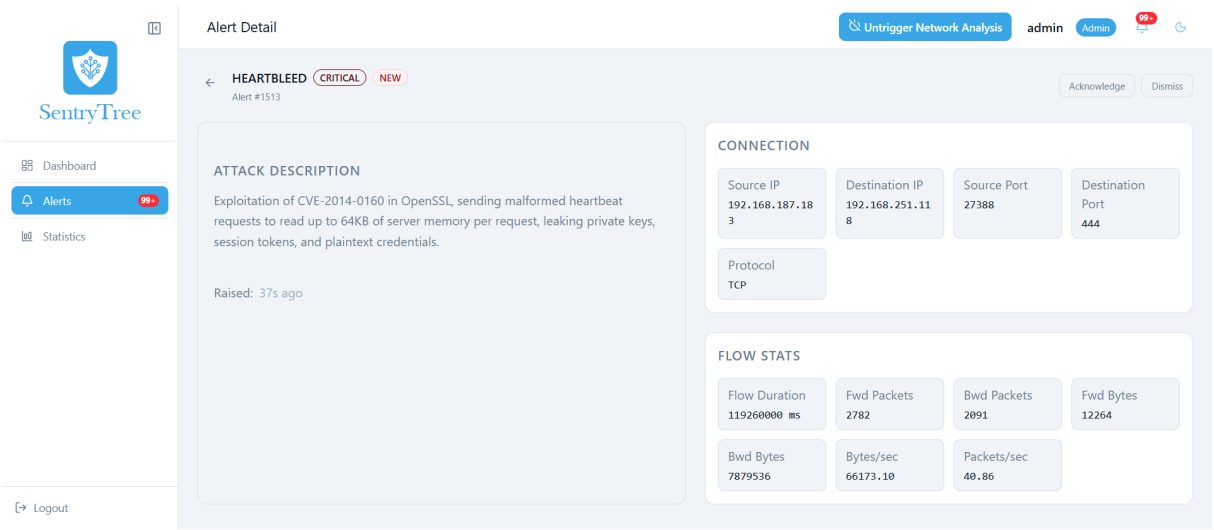


FIGURE 6.14 – Alert details interface

Use case «Manage Alert Status» realization

The alert status management interface allows updating the status of an alert (e.g., new, acknowledged, resolved, dismissed), as shown in Figure 6.15.

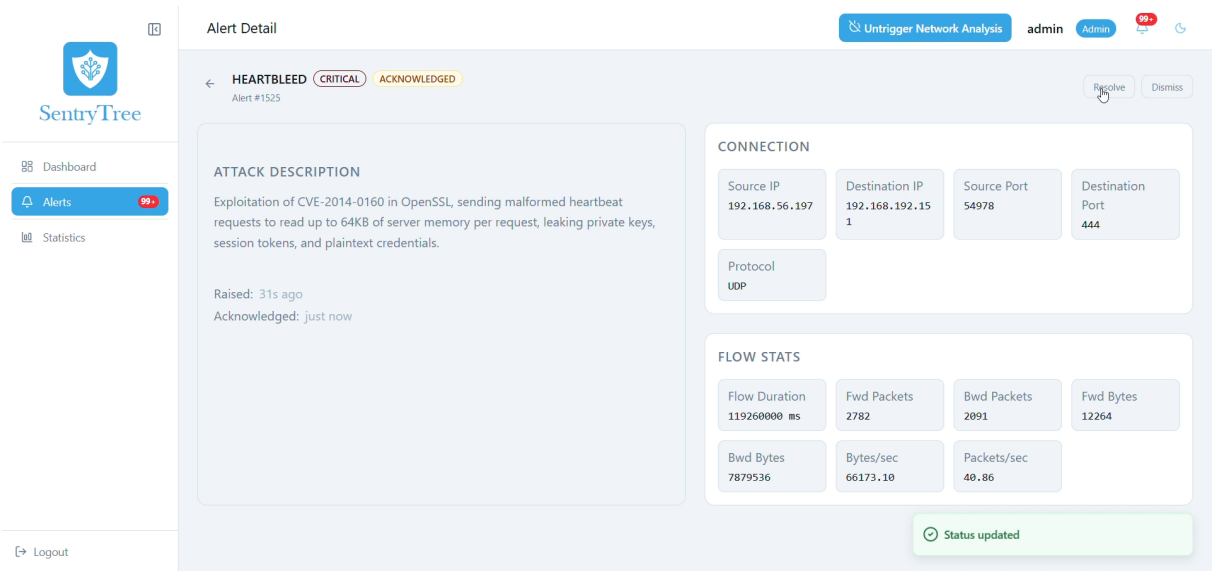


FIGURE 6.15 – Manage alert status interface

Use case «View Reports» realization

The reports interface allows the IT manager to browse and consult generated reports, as shown in Figure 6.16.

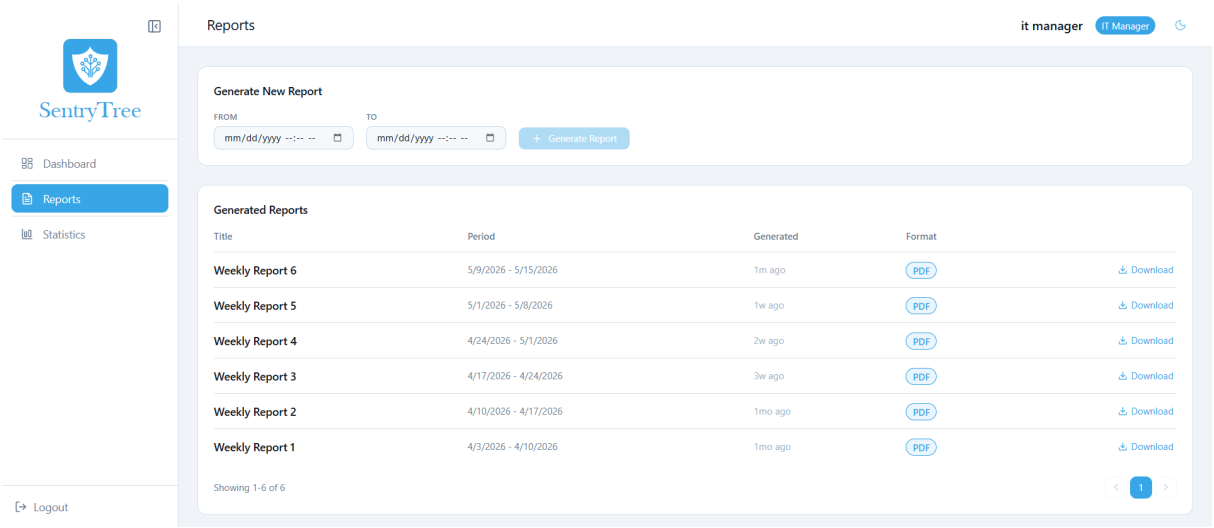


FIGURE 6.16 – Reports interface

Use case «Generate Report» realization

The report generation interface allows producing a new report by selecting a period and saving it for later consultation, as shown in Figure 6.17.

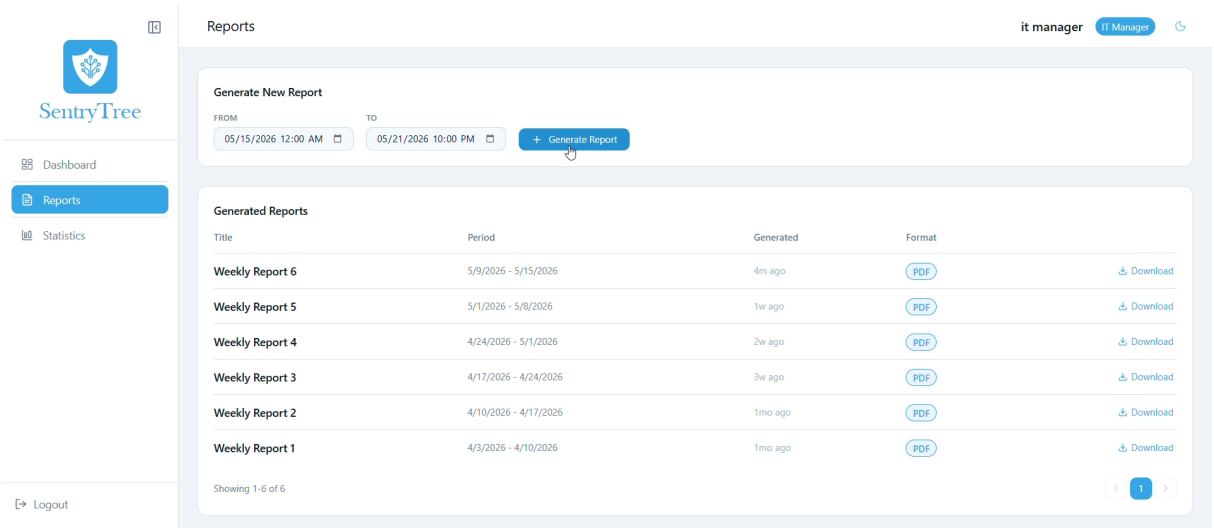


FIGURE 6.17 – Generate report interface

Use case «Export Report» realization

The export interface enables downloading a report in PDF format for sharing or archiving, as shown in Figure 6.18.

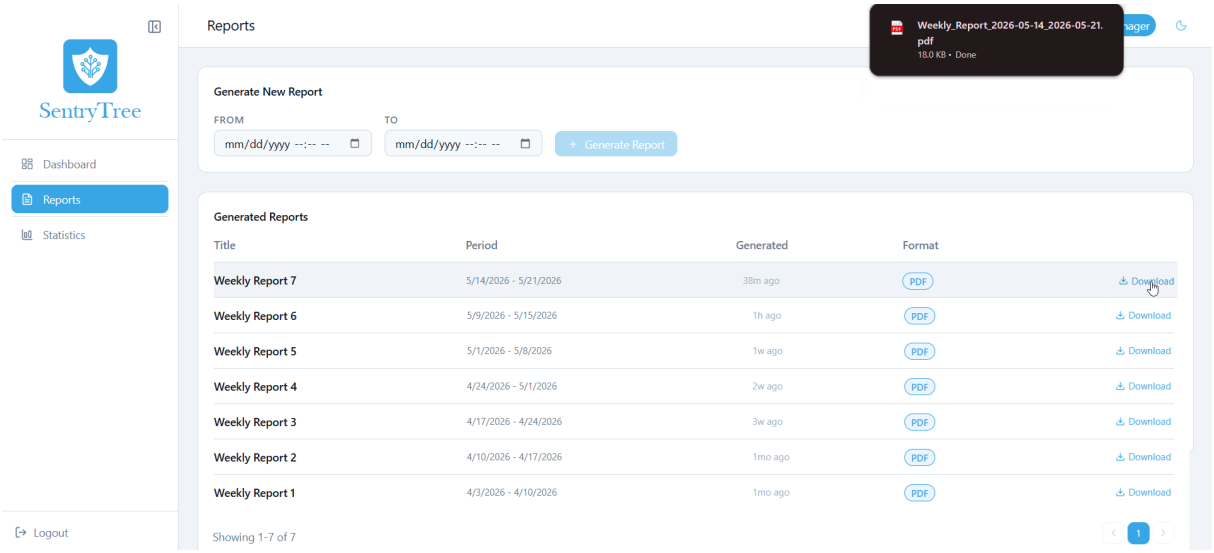


FIGURE 6.18 – Export report interface

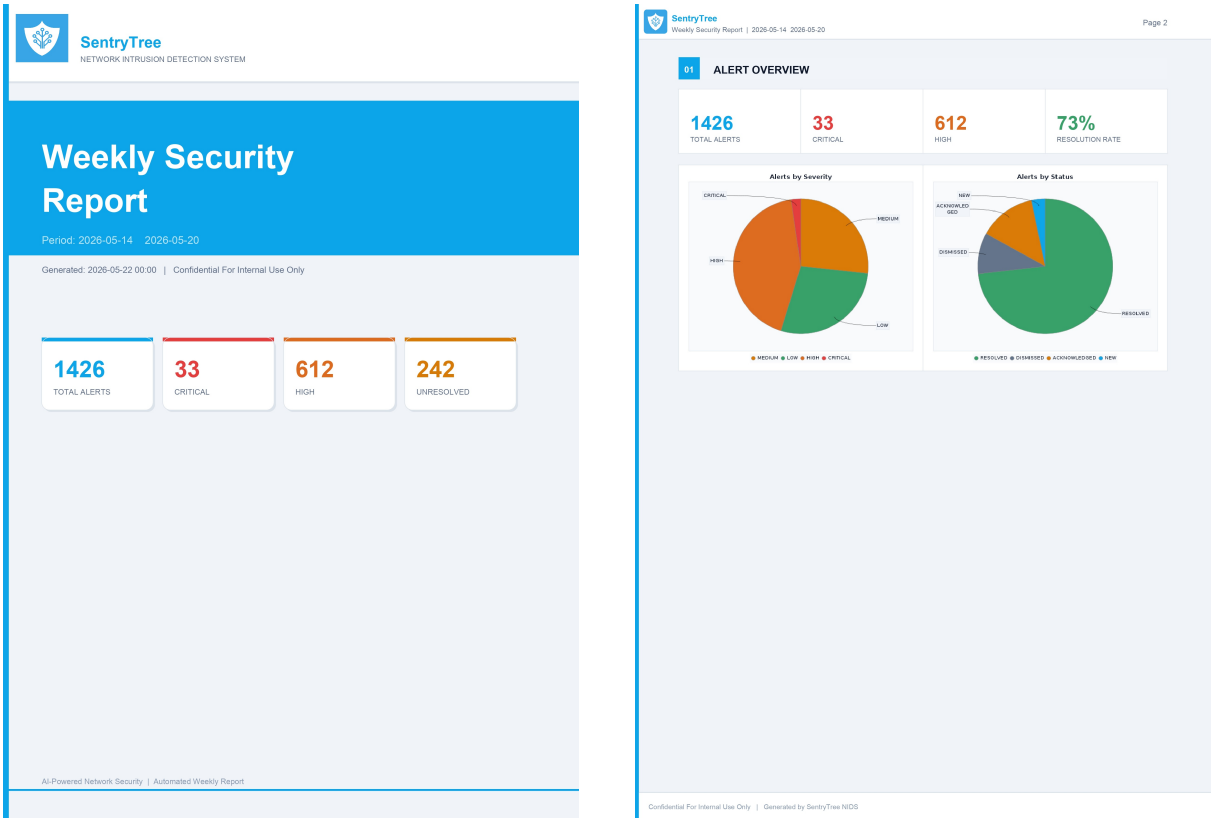


FIGURE 6.19 – Weekly report (pages 1–2)

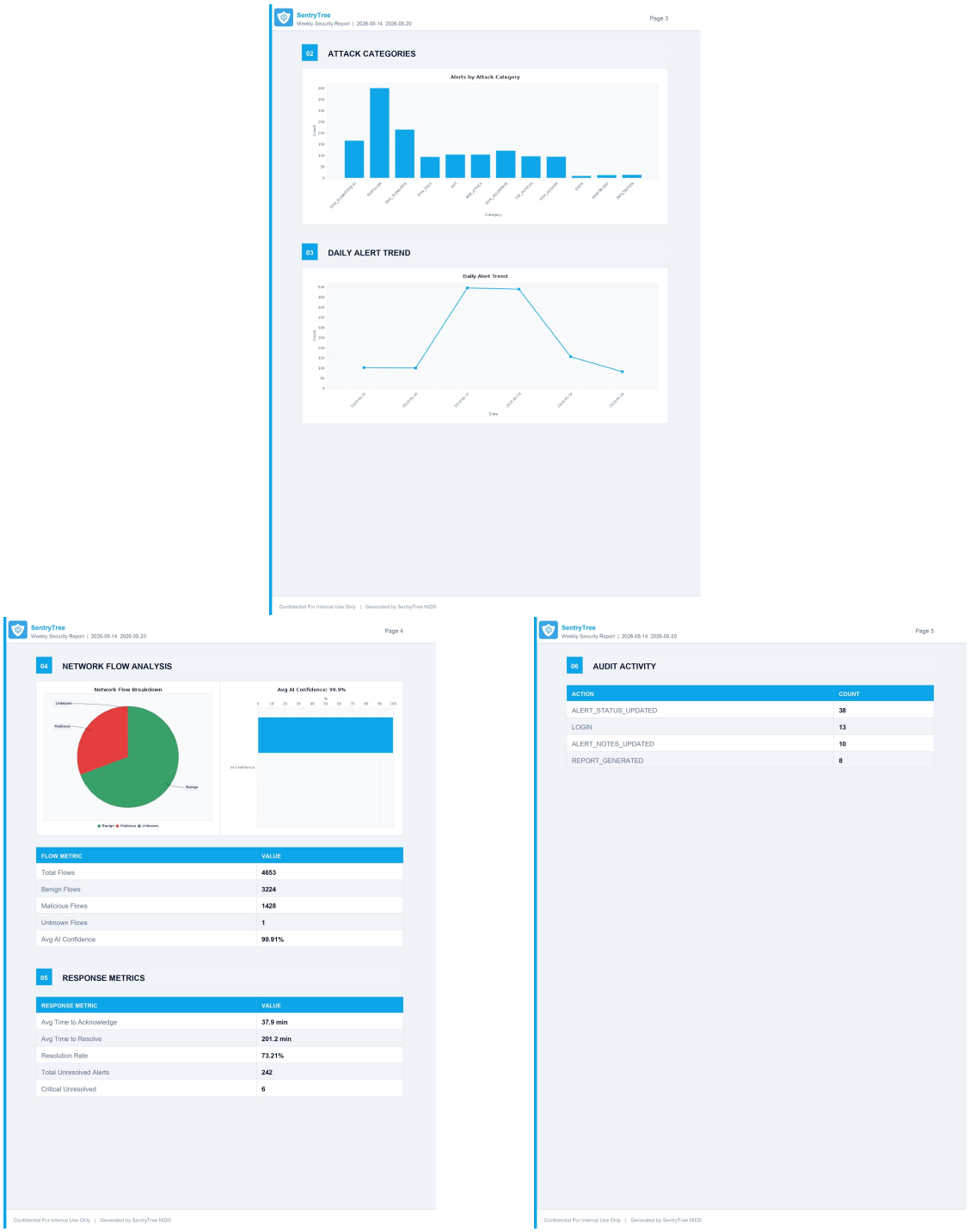


FIGURE 6.20 – Weekly report (pages 3–5)

6.6 Conclusion

With this final chapter, we conclude the design and implementation of the AI-based Network Intrusion Detection System. We presented the model integration through a FastAPI inference service, the containerized deployment using Docker and Docker Compose, and the main user interfaces covering the core functionalities of the platform. Finally, we outlined the development tools and technologies used across the different components of the system.

General Conclusion

This project focused on building an AI-based intrusion detection system.

We first defined the project context and justified the choice of an AI-driven approach by highlighting the weaknesses of traditional NIDS systems. We then specified the system's requirements, identified the actors, and modeled the system, before moving to the data understanding and preparation, which proved to be one of the most demanding phases of the project. Finally, we carried out the modeling, evaluation, and implementation steps.

Among the most important findings of this project was that intrusion detection performance depends as much on data quality as on algorithmic choice. The CIC-IDS2017 dataset offered a rich and realistic basis for experimentation, but it also required extensive inspection and preprocessing due to corrupted values, duplicated records, contradictory samples, and features affected by extraction issues. This confirmed that in any data science project, the reliability of the input data is often a decisive factor in the quality of the final model.

If time had permitted, we would have implemented the transformation pipeline that converts raw PCAP files captured in real time into a ready-for-prediction dataset row.

As a future extension, the most promising direction would be to introduce a second anomaly detection layer in addition to the current classifier. While the supervised model focuses on a fixed set of known attack categories defined by the dataset, an anomaly detection component could learn normal network behavior and identify deviations that may correspond to previously unseen attacks, including zero-day attacks. This could be further enhanced by integrating a deep learning layer trained on a suitable dataset to capture more complex patterns in network traffic. Together, this would enable a more robust hybrid architecture combining supervised classification, anomaly detection, and deep learning.

Overall, this project provided a significant learning experience that strengthened our understanding of both data science and cybersecurity concepts, particularly how intrusion detection systems operate and how network traffic data is structured and analyzed.

Webography

[3] https://www.snort.org/. Accessed on: April 6, 2026.

[4] https://suricata.io/. Accessed on: April 6, 2026.

[9] https://github.com/ahlashkari/CICFlowMeter. Accessed on: April 23, 2026.

[11] https://www.heartbleed.com. Accessed on: May 4, 2026.

[19] https://owasp.org/www-community/attacks/xss/. Accessed on: May 4, 2026.

[20] https://owasp.org/www-community/attacks/SQL_Injection. Accessed on: May 4, 2026.

[21] https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html. Accessed on: May 12, 2026.

[35] https://www.python.org/. Accessed on: May 15, 2026.

[36] https://scikit-learn.org/. Accessed on: May 15, 2026.

[37] https://spring.io/projects/spring-boot. Accessed on: May 15, 2026.

[38] https://react.dev/. Accessed on: May 15, 2026.

[39] https://tailwindcss.com/. Accessed on: May 20, 2026.

[40] https://www.mysql.com/. Accessed on: May 15, 2026.

[41] https://www.apachefriends.org/. Accessed on: May 15, 2026.

[42] https://www.postman.com/. Accessed on: May 15, 2026.

[43] https://fastapi.tiangolo.com/. Accessed on: May 15, 2026.

[44] https://www.docker.com/. Accessed on: May 15, 2026.

[45] https://docs.docker.com/compose/. Accessed on: May 15, 2026.

[46] https://git-scm.com/. Accessed on: May 15, 2026.

[47] https://github.com/. Accessed on: May 15, 2026.

[48] https://code.visualstudio.com/. Accessed on: May 15, 2026.

[49] https://www.jetbrains.com/idea/. Accessed on: May 15, 2026.

Bibliography

[1] R. Wirth and Jochen Hipp. "CRISP-DM: Towards a standard process model for data mining". In: *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining* (Jan. 2000), pp. 4–7.

[2] John Erickson and Keng Siau. "Unified Modeling Language: The Teen Years and Growing Pains". In: vol. 8016. July 2013, pp. 3–6. ISBN: 978-3-642-39208-5. DOI: 10.1007/978-3-642-39209-2_34.

[5] Chris Jackson. *Network Security Auditing*. 800 East 96th Street, Indianapolis, IN 46240 USA: Cisco Press, 2010. ISBN: 978-1-58705-352-8.

[6] Mahbod Tavallaee et al. "A Detailed Analysis of the KDD CUP 99 Data Set". In: *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*. 2009, pp. 1–6. DOI: 10.1109/CISDA.2009.5356528.

[7] Nour Moustafa and Jill Slay. "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)". In: *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015. DOI: 10.1109/MilCIS.2015.7348942.

[8] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization". In: *ICISSP* (2018), pp. 108–116. DOI: 10.5220/0006639801080116.

[10] Arash Mahboubi et al. "The evolving threat landscape of botnets: Comprehensive analysis of detection techniques in the age of artificial intelligence". In: *Internet of Things* 33 (2025), p. 101728. ISSN: 2542-6605. DOI: https://doi.org/10.1016/j.iot.2025.101728. URL: https://www.sciencedirect.com/science/article/pii/S2542660525002422.

[12] Jon Postel and Joyce Reynolds. *File Transfer Protocol (FTP)*. RFC 959. Internet Engineering Task Force, Oct. 1985. URL: https://www.rfc-editor.org/rfc/rfc959.

[13] Tatu Ylonen and Chris Lonvick. *The Secure Shell (SSH) Protocol Architecture*. RFC 4251. Internet Engineering Task Force, Jan. 2006. URL: https://www.rfc-editor.org/rfc/rfc4251.

[14] Kadek Wikrama et al. "DDoS Attack Using GoldenEye, DAVOSET, and PyLoris Tools". In: *Jurnal CoreIT: Jurnal Hasil Penelitian Ilmu Komputer dan Teknologi Informasi* 9 (Dec. 2023), p. 43. DOI: 10.24014/coreit.v9i2.20020.

[15] Chandra Dash and Chipsy Jha. "Real-Time Slowloris Attack Detection and Mitigation with Machine Learning Techniques". In: *International Journal of Engineering Research and Technology* 13 (Sept. 2024).

[16] Suroto Suroto. “A Review of Defense Against Slow HTTP Attack”. In: 2017. URL: https://api.semanticscholar.org/CorpusID:55876678.

[17] K.S. Vanitha, S V UMA, and S.K. Mahidhar. “Distributed denial of service: Attack techniques and mitigation”. In: *2017 International Conference on Circuits, Controls, and Communications (CCUBE)*. 2017, pp. 226–231. DOI: 10.1109/CCUBE.2017.8394146.

[18] L. Bošnjak, J. Sreš, and B. Brumen. “Brute-force and dictionary attack on hashed real-world passwords”. In: *2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. 2018, pp. 1161–1166. DOI: 10.23919/MIPRO.2018.8400211.

[22] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd. Wiley-Interscience, 2006. ISBN: 978-0-471-24195-9.

[23] Arnaud Rosay et al. “Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017”. In: *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP 2022)*. SCITEPRESS, 2022, pp. 25–36. ISBN: 978-989-758-553-1. DOI: 10.5220/0010774000003120.

[24] Yihua Liao and Rao Vemuri. “Use of K-Nearest Neighbor classifier for intrusion detection”. In: *Computers & Security* 21 (Oct. 2002), pp. 439–448. DOI: 10.1016/S0167-4048(02)00514-X.

[25] Azidine Guezzaz et al. “A Reliable Network Intrusion Detection Approach Using Decision Tree with Enhanced Data Quality”. In: *Security and Communication Networks* 2021.1 (2021), p. 1230593. DOI: https://doi.org/10.1155/2021/1230593. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1155/2021/1230593. URL: https://onlinelibrary.wiley.com/doi/abs/10.1155/2021/1230593.

[26] Arnaud Rosay et al. “Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017”. In: *International Conference on Information Systems Security and Privacy*. 2022. URL: https://api.semanticscholar.org/CorpusID:246885520.

[27] Jerome H. Friedman. “Greedy function approximation: A gradient boosting machine.” In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. DOI: 10.1214/aos/1013203451. URL: https://doi.org/10.1214/aos/1013203451.

[28] Bobba Brao and Kailasam Swathi. “Fast kNN Classifiers for Network Intrusion Detection System”. In: *Indian Journal of Science and Technology* 10 (Apr. 2017), pp. 1–10. DOI: 10.17485/ijst/2017/v10i14/93690.

[29] Vincent Zibi Mohale and Ibidun Christiana Obagbuwa. “Evaluating machine learning-based intrusion detection systems with explainable AI: enhancing transparency and interpretability”. In: *Frontiers in Computer Science* Volume 7 - 2025 (2025). ISSN: 2624-9898. DOI: 10.3389/fcomp.2025.1520741. URL: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1520741.

[30] Mohammed Tarek Abdelaziz et al. “Enhancing Network Threat Detection with Random Forest-Based NIDS and Permutation Feature Importance”. In: *Journal of Network and Systems Management* 33.2 (Oct. 2024). ISSN: 1573-7705. DOI: 10.1007/s10922-024-09874-0. URL: http://dx.doi.org/10.1007/s10922-024-09874-0.

[31] Maya Hilda Lestari Louk and Bayu Adhi Tama. "Revisiting Gradient Boosting-Based Approaches for Learning Imbalanced Data: A Case of Anomaly Detection on Power Grids". In: *Big Data and Cognitive Computing* 6.2 (2022). ISSN: 2504-2289. DOI: 10.3390/bdcc6020041. URL: https://www.mdpi.com/2504-2289/6/2/41.

[32] David M. W. Powers. "Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation". In: *Journal of Machine Learning Technologies* 2.1 (2011), pp. 37–63.

[33] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[34] Marina Sokolova and Guy Lapalme. "A Systematic Analysis of Performance Measures for Classification Tasks". In: *Information Processing & Management* 45.4 (2009), pp. 427–437.